SHERLOCK

Security Review For Promis

Collaborative Audit Prepared For: **Promis**
Lead Security Expert(s): **eeyore**
ParthMandale
Date Audited: **July 15 - August 1, 2026**

Introduction

Promis is a platform for earning yield backed by real cash flow.

It issues two tokens:

- ProUSD is a liquid, yield-bearing token. Deposit stablecoins to receive ProUSD, whose exchange rate appreciates as yield accrues. At launch, ProUSD's yield comes entirely from established on-chain lending markets. It can be redeemed back to stablecoins at any time.
- ProUSD+ is a fixed-term position that earns a fixed rate over a set lock period. ProUSD+ capital funds revenue-based financing (RBF) for US small businesses through a licensed financing partner, so the yield derives from real business revenue rather than from token emissions or market speculation.

Both products are designed around independent verification. Reserves and financing data are reconciled daily by an independent verification provider and attested on-chain, so the backing of each position can be verified rather than assumed.

Scope

Repository: promis-finance/promis-contracts

Audited Commit: 4880eb585054984adccc0239341b0f2ba156cf15

Final Commit: 05d4f111e9273efbabc4a37374ee82a813467ed5

Files:

- contracts/core/ProTokenOperations.sol
- contracts/core/ProTokenSettings.sol
- contracts/core/ProToken.sol
- contracts/core/ProTokenUnmintHandler.sol
- contracts/core/state/ProTokenOperationsState.sol
- contracts/core/state/ProTokenSettingsState.sol
- contracts/core/state/ProTokenUnmintHandlerState.sol
- contracts/core/state/YAssetOperationsHandlerState.sol
- contracts/core/types/ProTokenOperationsTypes.sol
- contracts/core/types/ProTokenSettingsTypes.sol
- contracts/core/types/ProTokenUnmintHandlerTypes.sol
- contracts/core/types/YAssetOperationsHandlerTypes.sol
- contracts/core/YAssetOperationsHandler.sol

- contracts/oracles/interfaces/IOracleChainlinkPushAdaptor.sol
- contracts/oracles/OracleChainlinkPushAdaptor.sol
- contracts/oracles/state/OracleChainlinkPushAdaptorState.sol
- contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol
- contracts/vaults/ProTokenPlus/ProTokenPlus.sol
- contracts/vaults/ProTokenPlus/state/ProTokenPlusState.sol
- contracts/vaults/ProTokenPlus/state/StrategyVaultState.sol
- contracts/vaults/ProTokenPlus/StrategyVault.sol
- contracts/vaults/ProTokenPlus/types/ProTokenPlusTypes.sol
- contracts/yields/AaveV3YieldHandler.sol
- contracts/yields/MorphoYieldHandler.sol
- contracts/yields/state/AaveV3YieldHandlerState.sol
- contracts/yields/state/MorphoYieldHandlerState.sol

Final Commit Hash

05d4f111e9273efbabc4a37374ee82a813467ed5

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	8	31

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: `claimGrowth` is documented as admin-only but is implemented `onlyAdminOrOperator` with a caller-supplied recipient [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/27

Summary

`IStrategyVault.claimGrowth` is documented as admin-only in two places, but the implementation is gated `onlyAdminOrOperator`. Unlike the sibling `claimYield`, the recipient is also a caller-supplied `to` rather than the admin-set `yieldRecipient`.

Vulnerability Detail

The interface states the restriction twice:

```
/// @notice Admin-only: withdraw accrued proUSD appreciation yield.
/// @dev Only admin. Settles first to capture appreciation up to the current price,
```

The implementation carries `@inheritdoc IStrategyVault`, so this is the NatSpec reviewers and tooling see, yet the modifier is `onlyAdminOrOperator`.

The gap is not purely cosmetic. `claimYield`, the adjacent claim function with the same gate, pins its destination to `yieldRecipient` and reverts `YieldRecipientNotSet` otherwise. `claimGrowth` has no such pin, and transfers to whatever `to` the caller passes, checked only against `address(0)`. The operator can therefore move all of `growthProUSD` to an arbitrary address.

Impact

The operator role holds an undocumented privilege.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L413-L417

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/interfaces/IStrategyVault.sol#L239-L249

Tool Used

Manual Review

Recommendation

Decide which side is authoritative and make the other match: either restrict the modifier to `onlyAdmin`, or update the NatSpec to document operator access.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/b2dd2c5
- Modified NatSpec to match the actual modifier

0xklapouchy

Fix confirmed in b2dd2c5. Operator access is intentional.

Issue M-2: Protocol-owned unmint fees become redeemable by remaining proUSD holders [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/40

Summary

`ProTokenOperations` deducts `unmintFeePer` from a user's redemption and says that the protocol keeps the fee. However, the fee is not transferred to a protocol recipient or recorded separately. The full proUSD amount is burned, only the net yAsset amount is paid or queued, and the fee remains mixed with proUSD backing.

If the next price checkpoint uses gross backing divided by live supply, that fee increases the proUSD price and becomes redeemable by the remaining holders.

With an allowed `1%` fee, a `$1 million` redemption from a `$10 million` pool leaves a `$10,000` fee. After the fee is included in the next checkpoint, a passive holder that initially owned only `1%` of supply can redeem `$110`, or `1.10%` of the fee, while retaining a `$100,000` gross proUSD claim.

Vulnerability Detail

The fee exists only as a local variable. After it is subtracted, the contract burns the user's complete proUSD input and passes the reduced amount to `payOut()` or `createUnmintRequest()`. There is no fee recipient, fee ledger, or fee-specific collection step.

`YAssetOperationsHandler.getYAssetInfo()` reports all idle and venue-held yAssets as backing. `ProToken.updateUSDPrice()` accepts an absolute price without any fee-exclusion input. Therefore, a gross-backing checkpoint treats the retained fee like Aave/Morpho yield and allocates it to current holders.

The source explicitly says `Fee stays on the protocol (protocol keeps it)`, while the protocol documentation attributes proUSD appreciation to Aave/Morpho yield. No supplied specification assigns unmint fees to remaining holders.

A simple example demonstrates the issue:

1. Backing and supply are `10,000,000`. The passive holder owns `100,000 proUSD` (1%).
2. Another user redeems `1,000,000 proUSD` with a `1%` fee. The user receives `990,000 USDC`, and `10,000 USDC` remains as the protocol fee.
3. Supply becomes `9,000,000 proUSD` and backing is `9,010,000 USDC`. After the normal cooldown, the exact gross-backing checkpoint is `$1.001111111111111111`.
4. The passive holder redeems only the shares representing its fee-derived appreciation. After its own 1% fee, it receives exactly `110 USDC`.

5. The holder retains `99,889.012208657047735837 proUSD`, representing its original `$100,000` gross claim at the checkpoint. The `$111.111111` gross windfall also exceeds the protocol's `$100` minimum withdrawal.

The 1% fee is well below the contract's 10% maximum and keeps the example at a realistic scale. A lower fee changes only the balances required to demonstrate the same accounting issue; every retained fee is still included in holder backing unless it is separated before the checkpoint.

Impact

The protocol loses `110 USDC`, or `1.10%` of the `10,000 USDC` protocol fee, to a holder that did not pay that fee. This is a direct loss of protocol fee revenue.

Code Snippet

In `contracts/core/ProTokenOperations.sol`, the fee is subtracted but never transferred or recorded:

```
if (yAssetSettings.settings.unmintFeePer > 0) {
    // Calculate the fee amount using wad precision (1e18)
    uint256 feeAmount = (yAssetToReceiveAmount *
        yAssetSettings.settings.unmintFeePer) / USD_PRECISION;

    // Reduce the amount to unmint by the fee amount
    yAssetToReceiveAmount -= feeAmount;

    // Fee stays on the protocol (protocol keeps it)
}

// burn the pro tokens from the user
IProToken(proTokenInfo.proToken).burn(address(this), _amount);
```

Only the reduced amount is used for settlement:

```
try yOps.payOut(_recipient, yAssetToReceiveAmount) {
    paidInstant = true;
} catch {
    // fall through to the queue path below
}
```

```
uint256 handlerRequestId = IProTokenUnmintHandler(
    proTokenInfo.proTokenUnmintHandler
).createUnmintRequest(_recipient, _yAsset, yAssetToReceiveAmount);
```

In `contracts/core/YAssetOperationsHandler.sol`, all managed balances are reported without excluding fees:

```
totalAmount += handlerBalance;

// add the unallocated balance
totalAmount += IERC20(yAsset).balanceOf(address(this));
```

In `contracts/core/ProToken.sol`, the supplied price is stored without fee settlement:

```
usdPrice = _price;
lastPriceUpdateAt = block.timestamp;
```

Recommendation

Separate the fee from holder backing when it is charged. Either transfer `feeAmount` to a configured protocol recipient or record it in an `accruedProtocolFees[yAsset]` ledger that is excluded from every proUSD price checkpoint and can be claimed only by the protocol.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/7e45456
- UnmintFees are now accounted for in YAssetOperationsHandler and are now collectable

Parth0x108

correctly Fixed, just add `collectFee()` inside `IYAssetOperationsHandler.sol` for consistency.

promisque

- https://github.com/promis-finance/promis-contracts/commit/998c4f0e07ad5ac9ccce4e717071258c708f259d
- Added collectFee() inside *IYAssetOperationsHandler.sol*

Issue M-3: Minting proUSD at the stored price lets new holders capture previously accrued yield [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/41

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`ProTokenOperations` calculates newly minted proUSD from the deposited asset's USD value and the last stored proUSD price. It does not use the backing and `totalSupply()` that exist immediately before the mint.

If Aave or Morpho has accrued yield since the last price update, a new holder is therefore minted too many shares and receives part of the yield earned before entering. Updating to the exact NAV afterward keeps the protocol solvent, but it cannot restore the yield already assigned to the new shares.

Consider a conservative scenario with a 6-decimal stablecoin, a 5% annualized rate, a 0.1% unmint fee, 100% Aave allocation, and a 23-hour accrual window matching the contract's default operator-update cooldown. A `$1,000,000` entrant receives `$119.224782` of incumbent yield while retaining a proUSD position worth `$1,000,000` at the published price.

Vulnerability Detail

The mint calculation is effectively:

```
minted proUSD = credited deposit value / stored proUSD price
```

Because the calculation does not read current backing or supply, the stored price can be lower than the actual backing per existing proUSD whenever yield has accrued between price updates.

The mint approval does not prevent this. It contains `minAmountOut`, which is a lower bound chosen by the minter, but no valuation epoch or maximum number of shares. Both the mint and later redemption use valid ordinary authority approvals.

The following example shows the value transfer:

1. Existing holders own `10,000,000` proUSD backed by `10,000,000 USDC`, all routed through `AaveV3YieldHandler`.
2. At a 5% annualized rate, 23 hours of linear accrual produces `1,312.785388 USDC` after the 6-decimal token truncation:

```
10,000,000 * 5% * 23 / (365 * 24) = 1,312.785388 USDC
```

3. A new approved user deposits `1,000,000 USDC`, equal to 10% of incumbent principal. Because the stored proUSD price is still `$1`, the contract mints `1,000,000` proUSD.
4. The price operator then publishes the exact post-entry NAV:

```
(10,000,000 + 1,312.785388 + 1,000,000) / 11,000,000
= $1.000119344126181818
```

5. At that exact price, the entrant's shares are worth `$1,000,119.344126`. The extra `$119.344126` came entirely from yield that accrued before the entrant joined.
6. The mint is already finalized before the price update. The entrant later obtains a fresh unmint approval and burns only the shares representing the windfall. The gross payout is `119.344126 USDC`; the 0.1% fee is `0.119344 USDC`; and the entrant receives `119.224782 USDC`.
7. At the published stored price, the remaining entrant position is worth exactly `$1,000,000`, while handler backing decreases by exactly `119.224782 USDC`.

The realized amount is:

```
119.224782 / 1,312.785388 = 9.0818% of incumbent yield
```

This exceeds the `$100` withdrawal minimum. No false price, malicious signer, attacker donation, or multi-day missed checkpoint is required.

These are conservative illustrative values rather than claimed live configuration. The assumed 5% annualized rate is below the 6-7% combined Aave/Morpho yield publicly described by Promis, which also confirms USDC/USDT support (Promis products, Promis terms). Aave confirms that supplied balances accrue at variable market rates (Aave supply documentation).

Impact

Existing proUSD holders directly lose yield that was already part of protocol backing before the entrant joined. In the example, the entrant receives `119.224782 USDC`, equal to `9.0818%` of the affected interval's incumbent yield, and the handler's backing falls by the same amount.

This is a transfer of underlying stablecoin from incumbent holders to the new holder, not merely an incorrect displayed price or temporary accounting difference.

Code Snippet

In `contracts/core/ProTokenOperations.sol`, the mint proof does not bind a valuation epoch or maximum share output:

```
bytes32 private constant MINT_PROOF_TYPEHASH = keccak256("MintProof(uint256
↪  requestId,address user,address receiver,address yAsset,uint256 amount,uint256
↪  minAmountOut,uint256 deadline,uint8 proofKind)");
```

The proUSD value used during minting comes only from the stored price:

```
function _getUsdRepresentationProToken(
    address _proToken,
    uint256 _amount,
    uint8 _decimals
) internal view returns (UsdRepresentation memory usdRepresentation) {
    // get price directly from pro token contract
    usdRepresentation.assetUSD = IProToken(_proToken).getUSDPrice();
    usdRepresentation.assetAmountUSD =
        (_amount * usdRepresentation.assetUSD) /
        (10 ** _decimals); // come as 18 decimals
}
```

The deposit value is divided by that stored price without reading backing or `totalSupply()`:

```
uint256 cappedYAssetAmountUSD = (yAssetAmount *
    cappedYAssetUSD) / (10 ** yAssetDecimals);
return
    (cappedYAssetAmountUSD * USD_PRECISION) /
    proTokenUsdRep.assetAmountUSD;
```

A holder can redeem only the windfall amount, so the fee applies only to that partial payout:

```
if (yAssetSettings.settings.unmintFeePer > 0) {
    uint256 feeAmount = (yAssetToReceiveAmount *
        yAssetSettings.settings.unmintFeePer) / USD_PRECISION;

    yAssetToReceiveAmount -= feeAmount;
}

IProToken(proTokenInfo.proToken).burn(address(this), _amount);
```

Tool Used

Manual Review

Recommendation

When supply is nonzero, mint from the protocol's pre-deposit equity rather than the last stored price alone:

```
shares = deposit value * totalSupply / pre-deposit holder equity
```

Use conservative live backing after subtracting unpaid senior liabilities and segregated non-holder balances. The valuation and mint must be atomic, or the approval must bind a valuation epoch and `maxSharesOut` and revert when that state changes.

Updating NAV only after minting is insufficient because the excess shares are already included in the new supply.

Discussion

promisque

- Disputed
- Requesting downgrade from High to Low
- Reason: The proUSD price is authority-published with a minimum 23-hour cooldown between updates; based on off-chain NAV computation; the staleness window between updates is a deliberate operational property, not a code defect. The finding's recommended fix (atomic live-backing mint formula) would require on-chain NAV computation at mint time, coupling minting to venue readability and removing the authority-controlled pricing model the protocol is designed around. The unmint fee creates a cost floor for round-trip extraction; if the fee exceeds the typical per-window NAV appreciation step, the scenario is economically self-preventing under real configuration.

As a structural mitigation planned for a future iteration, we intend to replace the single published price with a delayed linear interpolation between two realized NAVs: the on-chain price will lag one window behind and ramp linearly from the previous realized NAV to the most recent realized NAV over the update window. Since the one-window delay makes both endpoints already-realized (never projected) values, the interpolated price stays bounded by known-good NAVs and never leads actual backing.

0xklapouchy

Agreed on a downgrade, to Medium rather than Low.

The unmint fee is the argument that carries it, and it stands on its own with no code change: one extraction nets roughly 1.2 bps of the deposit, and repeating it requires a full exit costing 10 bps of principal. That makes it a one-time entry bonus per depositor rather than a repeatable drain, which is why it is not High. It is also bounded by inflow rather than by TVL.

One note on the planned mitigation: we reviewed the linear interpolation branch and no longer recommend it. It removes the timeable mint-before-update, redeem-after

variant, but the gap between published price and actual backing goes from resetting at every update, averaging about half a window, to sitting at a full window constantly. So it roughly doubles what each depositor takes from existing holders. It is only an improvement if minting also moves to futurePrice while redemption stays on the interpolated price, and that change makes getUSDPrice() directional across ProTokenOperations, ProTokenPlusOperations and StrategyVault.

Issue M-4: Full-value proUSD issuance into an uncovered Aave deficit lets existing aToken holders consume fresh backing [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/42

Summary

`ProTokenOperations` calculates proUSD from the full value of a user's deposit before routing the deposit to its yield handlers. It does not verify that the selected route adds an equal amount of conservative backing before minting the already-calculated proUSD.

This assumption fails for an Aave V3.3 reserve with an uncovered deficit. Aave can accept the complete supply and mint the complete nominal aToken amount without reducing the deficit. A pre-existing aToken holder can then burn its own impaired aTokens and withdraw the cash just supplied by Promis, leaving Promis with nominal aTokens but no corresponding marginal backing.

Vulnerability Detail

Aave V3.3 records realized bad debt in `reserve.deficit`. Its validateSupply() checks the reserve flags and supply cap, but not the deficit. executeSupply() therefore transfers the full underlying and mints the full nominal aToken amount while the deficit remains unchanged. Withdrawals burn the caller's aTokens against the reserve's shared cash; Aave does not reserve later deposits for the later supplier. The deficit is reduced only through the separate permissioned coverage process described in the Aave V3.3 deficit specification.

Promis neither checks this deficit before allocation nor measures the conservative backing added by the route. `AaveV3YieldHandler.getBalance()` also reports the handler's full nominal aToken balance, even when those receipts are impaired.

The mint approval does not close the issue because it binds the request amount and output floor, but not the Aave Pool, allocation, reserve deficit, or coverage state. A still-valid approval issued before the deficit is recorded can therefore be finalized after a liquidation creates it.

A concrete sequence is:

1. Incumbents hold `10,000,000 proUSD` backed by `$20,000,000` of healthy assets. The stored `$2` NAV is exact, and Promis initially has no exposure to the affected Aave reserve.
2. An external holder in an Aave reserve with `50,000,000` external aTokens has a still-valid approved `5,000,000 USDC` Promis mint request. A liquidation then records a `35,300,000 USDC` reserve deficit and leaves `14,700,000 USDC` of old cash.

3. The holder withdraws the old cash, leaving Aave with zero cash, no collectible borrower debt, and nominal aToken supply equal to the recorded deficit. The remaining aTokens consequently have zero currently executable reserve backing in this state.
4. The holder finalizes the mint. Promis values the complete deposit at `$5,000,000`, routes all of it to Aave, receives `5,000,000` nominal aTokens, and mints `2,500,000 proUSD` at the exact `$2` price.
5. The holder burns `5,000,000` of its pre-existing aTokens and withdraws the exact `5,000,000 USDC` introduced by Promis. Its USDC seed is restored, while Promis retains only the impaired nominal receipts.
6. Conservative Promis backing remains `$20,000,000`, but supply is now `12,500,000 proUSD`. NAV is therefore `$1.60`. The incumbent position falls from `$20,000,000` to `$16,000,000`, while the newly minted position is worth `$4,000,000`.

This does not assume that impaired aTokens are universally worthless. The demonstrated state establishes zero currently withdrawable reserve value; any secondary-market price or expected future Aave coverage is an opportunity cost to the holder. Full deficit coverage before finalization makes the sequence value-neutral.

Impact

Fresh user capital can recapitalize pre-existing external Aave claims while Promis issues proUSD as though the complete deposit remained in its backing. This immediately dilutes incumbent holders and makes the new proUSD underbacked relative to its issuance price.

In the example, incumbents lose `$4,000,000`, or 20% of their backing value, and the new holder receives a proUSD position worth the same amount while recovering its complete cash seed. The loss is bounded by the unsafe Aave allocation, uncovered deficit, and available external aToken inventory.

The path requires a material uncovered deficit, an active supply-enabled reserve with cap headroom, nonzero Promis allocation, sufficient unencumbered external aTokens, and no complete coverage or operational rejection before finalization. These specific external conditions make this a Medium-severity, deployment-conditional issue; no current live deficit is assumed.

Code Snippet

In `contracts/core/ProTokenOperations.sol`, the proof does not bind the downstream venue or its solvency state:

```
bytes32 private constant MINT_PROOF_TYPEHASH = keccak256("MintProof(uint256
↪  requestId,address user,address receiver,address yAsset,uint256 amount,uint256
↪  minAmountOut,uint256 deadline,uint8 proofKind)");
```

The complete deposited value is converted into proUSD:

```
uint256 cappedYAssetAmountUSD = (yAssetAmount *
    cappedYAssetUSD) / (10 ** yAssetDecimals);
return
    (cappedYAssetAmountUSD * USD_PRECISION) /
    proTokenUsdRep.assetAmountUSD;
```

The precomputed amount is minted after routing, without validating the conservative backing actually added:

```
IERC20(_yAsset).safeTransfer(
    settings.yAssetSettings.settings.yOperationsHandler,
    _yAmount
);

IYAssetOperationsHandler(
    settings.yAssetSettings.settings.yOperationsHandler
).distributeYAsset(_yAmount);

IProToken(settings.proToken).mint(_recipient, _mintAmount);
```

In `contracts/core/YAssetOperationsHandler.sol`, each allocation is deposited at face value:

```
IERC20(yAsset).forceApprove(handlerAddr, allocationAmount);
IYieldProtocolHandler(handlerAddr).depositYieldAsset(allocationAmount);
```

In `contracts/yields/AaveV3YieldHandler.sol`, the handler supplies without checking the reserve deficit and reports only the nominal aToken balance:

```
IERC20(asset).forceApprove(pool, amount);
IPool(pool).supply(asset, amount, address(this), 0);
```

```
return IAToken(aTokenAddress).balanceOf(address(this));
```

Tool Used

Manual Review

Recommendation

Before every Aave allocation, query `getReserveDeficit(asset)` and reject or reroute the allocation while any material deficit remains uncovered. Perform this check during mint finalization so an approval issued in an earlier reserve state cannot bypass it.

Mint proUSD only after verifying that conservative post-route backing increased by at least the value represented by the new shares. Nominal aToken issuance or `balanceOf()` growth is not sufficient evidence of backing. Also make aggregate Aave valuation deficit-aware, but do not rely on valuation alone because it would reveal the loss only after unsafe issuance.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/e0ec35c
- Allocation now checks if handler accepts an allocation. Each handler has it's own term for accepting allocation. For aave, is to have the deficit within tolerance set for example
- AllocationSkipped event is emitted for non-accepting handlers

Parth0x108

The main issue is fixed, but the health check should fail closed when Aave data cannot be verified, and zero tolerance should reject every nonzero deficit without basis-point rounding. Once these cases and the upgrade-layout concerns are corrected, the issue can be marked fully fixed.

promisque

- https://github.com/promis-finance/promis-contracts/commit/8417bc46a9b2910278453cda7dbac15e9652a3af
- Fixed fail closed and zero tolerance concerns
- https://github.com/promis-finance/promis-contracts/commit/e0ec35c
- Upgrade layouts were already fixed here

Parth0x108

The revised health check correctly fails closed when Aave data is unreadable, and zero tolerance now rejects every nonzero deficit, so the original issue path is blocked while tolerance remains zero.

Issue M-5: New deposits can be consumed by older underfunded unmint queues [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/44

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

A queued unmint is recorded as a fixed yAsset liability. If Morpho later suffers a loss, the queue can become larger than the protocol's remaining backing. The mint path still accepts a new deposit and mints proUSD using only the stored price, without checking the old queue liability or live net equity.

When the old queue is later funded from protocol backing, it can consume the new user's deposit and leave their newly minted proUSD partially or completely unbacked.

Vulnerability Detail

Assume an old queued redemption is owed `1000 USDC` and was fully backed when created. A later Morpho loss reduces live backing to `700 USDC`, but the fixed queue claim remains `1000 USDC`. The protocol therefore has a `300 USDC` deficit.

A new user can still deposit `300 USDC` and receive proUSD because minting considers only the deposit value and stored proUSD price. The new `300 USDC` is routed into Morpho, raising recoverable backing from `700` to `1000 USDC`. The operator then withdraws this pooled 1000 USDC to fund the old queue, which the old claimant claims, leaving no backing for the new user's proUSD.

The old claimant receives only the amount already promised. The issue is that the protocol admits a new depositor while old senior liabilities already exceed live backing. A proof signed before the loss remains usable because it does not bind the backing, queue amount, or loss state.

Impact

A normal new depositor can unknowingly recapitalize an older insolvent queue and lose principal. If the new deposit equals the inherited deficit, 100% of that deposit can be consumed.

The issue requires an unpaid queue, a later external loss, a separate mint, and routine queue funding from pooled backing. These conditions make the issue Medium severity.

Code Snippet

`contracts/core/ProTokenOperations.sol` calculates minted proUSD from the deposit value and stored proUSD price only:

```
uint256 cappedYAssetAmountUSD = (yAssetAmount *
    cappedYAssetUSD) / (10 ** yAssetDecimals);

return
    (cappedYAssetAmountUSD * USD_PRECISION) /
    proTokenUsdRep.assetAmountUSD;
```

The complete deposit is routed before the precomputed proUSD amount is minted:

```
IERC20(_yAsset).safeTransfer(
    settings.yAssetSettings.settings.yOperationsHandler,
    _yAmount
);

IYAssetOperationsHandler(
    settings.yAssetSettings.settings.yOperationsHandler
).distributeYAsset(_yAmount);

IProToken(settings.proToken).mint(_recipient, _mintAmount);
```

`contracts/core/ProTokenUnmintHandler.sol` already records and exposes the missing queue liability:

```
curBatch.totalAmount += amount;
unpaidQueuedLiability[yAsset] += amount;
```

```
function getUnpaidQueuedLiability(
    address yAsset
) external view returns (uint256) {
    return unpaidQueuedLiability[yAsset];
}
```

Tool Used

Manual Review

Recommendation

Before every ordinary and strategic mint, calculate conservative pre-deposit net equity:

```
net equity = live backing - unpaid queued liabilities
```

Reject minting whenever net equity is negative or below a configured positive safety buffer.

Minting should resume only after protocol-funded recapitalization or an explicit on-chain loss-allocation process. Users whose mint finalization is rejected by this check should also be able to recover their pending deposit.

Discussion

promisque

- Acknowledged
- Since this is a pool based protocol, this is intended. Newer deposits incase of slashing or loss for the protocol do go to the same shared pool.
- Minting can not be rejected since in a non-yield-generating environment, the backing always moves pricewise because of the asset pricing.
- A slashing event will be exceptional and loss of funds will be covered by the protocol

Parth0x108

Accepted risk rather than Fixing. Pooling does not remove the demonstrated behavior: when an older fixed queue exceeds live backing, a new deposit can be consumed servicing that liability because minting does not check liability-adjusted equity. Protocol coverage is a valid operational mitigation, but it must occur before further minting or queue settlement and is not currently enforced on-chain.

Issue M-6: The instant unmint path cannot recover once the backlog becomes non-zero [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/50

Summary

Instant redemption is gated on `unpaidQueuedLiability` being zero, and that same gate forces every unmint into the queue while it is not. Queued requests keep an unprocessed batch open, which keeps the backlog non-zero, which keeps the gate closed. A single failed payout therefore degrades the protocol to batch-only redemption for as long as unmint demand continues, regardless of how much liquidity the reserve holds.

Vulnerability Detail

The cycle is self-sustaining:

1. One instant payout fails for any reason, a batch opens, and `unpaidQueuedLiability` rises above zero.
2. The `backlog == 0` gate now forces every subsequent unmint into the queue, even with ample liquidity.
3. Those forced requests join the open batch, or open a new one once the previous window has elapsed.
4. `processNextUnmintBatch` reverts while a batch is inside its window, so an operator can clear every batch except the currently open one.
5. If that open batch holds even one request, the backlog stays above zero and step 2 repeats.

Recovery requires the operator to process a batch at the exact moment it matures with no request arriving first. The two conditions coincide precisely: a batch becomes processable when `createTimestamp + unmintBatchDuration` is reached, and the same instant is when a new request opens a fresh batch instead of joining it. Under steady demand the operator loses that race repeatedly, and the protocol never returns to instant settlement. Under quiet demand it self-heals after one empty window.

There is no lever to force recovery. Nothing allows an operator or admin to close or fund the currently open batch early, so the state cannot be cleared on demand.

The batch-selection logic indicates early processing was intended. It opens a new batch when `curBatch.processed` is true, but that clause is unreachable as written: a batch can only reach `processed` after its window has elapsed, and at that point the adjacent timestamp clause is already satisfied. The condition is only meaningful if a batch can be processed while still inside its window, so the creation path already accommodates early processing while `processNextUnmintBatch` forbids it.

Impact

A single failed payout permanently converts redemption from instant and trustless to queued and operator-dependent while demand continues.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L410-L418 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenUnmintHandler.sol#L171-L190

Tool Used

Manual Review

Recommendation

Drop the window check in `processNextUnmintBatch` so an operator can close and fund the open batch on demand. The creation path already handles that case through its `curBatch.processed` branch, which is otherwise unreachable, so this looks closer to the original intent.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/ecfda48
- Window check from batch process is dropped

0xklapouchy

Fix confirmed in ecfda48.

Issue M-7: `repay` credits the withdraw reserve at the stale ratchet price while minting at the live price [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/53

Summary

`_accrueGrowth` leaves `lastPrice` untouched whenever the live price is at or below it, since the ratchet only moves upward. `repay` then mints proUSD at the live price but values the result at that stale, higher `lastPrice`. During any price decline that has not yet been marked down, every repayment credits `withdrawBase` with more USD than the minted tokens are worth.

Vulnerability Detail

`repay` calls `_accrueGrowth` first, which returns early without advancing the ratchet when the live price has not risen. `strategicMint` then prices the mint against the live proUSD price, so a lower price produces more tokens. The reserve ledger is credited from a different price:

```
uint256 usdAmount = (proUSDMinted * lastPrice) / USD_PRECISION;
withdrawProUSD += proUSDMinted;
withdrawBase   += usdAmount;
```

With a yAsset worth 1000 USD, a live price of 1.0 and `lastPrice` of 1.2, the mint produces 1000 proUSD while `withdrawBase` is credited 1200 USD. Those tokens are worth 1000 at the live price, so 200 of reserve obligation is recorded that nothing backs.

The two sides of the vault disagree on which price to use. Deposits arrive through `give`, whose `worthBase` is computed by `_convertToBase` from `getUSDPrice()`, the live price. Repayments book at the ratchet. The discrepancy therefore opens only in one direction, and only while the live price sits below `lastPrice`.

The inflated figure is not inert. `surplusBase` is derived as `withdrawBase` minus `earmarkedWithdrawBase`, and that surplus is the sole bound on `claimYield`, `rotate` and `regive`. Phantom surplus is therefore claimable to `yieldRecipient` or movable into the strategist-borrowable deposit pool.

Pausing is not available as a mitigation. `StrategyVault` carries no pause modifier on any function, so `ProTokenSettings.pause()` does not reach `repay`, `claimYield`, `rotate` or `regive`. The exposure persists until an admin runs `markdownPrice` to re-anchor the ratchet.

Impact

Every repayment during an un-marked-down price decline inflates the withdraw reserve ledger, and the excess is claimable as yield or movable into the borrowable pool.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L360-L372

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L128-L143

Tool Used

Manual Review

Recommendation

Value the mint at the same price that produced it, so the two ledgers cannot diverge:

```
uint256 live = _tryGetPrice();
if (live == 0) revert PriceUnavailable();
uint256 usdAmount = (proUSDMinted * live) / USD_PRECISION;
```

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/55246c3
- Live price is now used in repay

0xklapouchy

Fix confirmed in 55246c3.

Issue M-8: `_accrueGrowth` banks the change in a pool's requirement rather than its actual excess, converting user backing into claimable growth [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/54

Summary

`_accrueGrowth` credits `growthProUSD` with `backingNeededLast - backingNeededNow`, the fall in what the base ledger requires. That equals the pool's true excess only while the pool is exactly backed. Two supported paths leave it short, and in both the next price rise repays a recorded loss rather than earning new appreciation. The code banks it as yield anyway, and `claimGrowth` pays it out.

Vulnerability Detail

The amount banked is derived from the obligation, never compared against what the pool actually holds:

```
uint256 freedBacking = backingNeededLast > backingNeededNow
    ? backingNeededLast - backingNeededNow
    : 0;
if (freedBacking > depositProUSD) freedBacking = depositProUSD;
```

The clamp bounds the amount by the pool's size rather than its surplus, so it catches a total wipe-out and nothing else. The reserve leg repeats the formula, as does the `claimableGrowth` view, so off-chain monitoring reports the inflated figure too.

Two paths break the exactly-backed premise.

A markdown that is not fully covered. `markdownPrice` lowers `lastPrice` to the live price and records `depositDeficit` and `withdrawDeficit` while leaving both token pools untouched. The interface states this plainly, that pools remain under-backed until cover lands, and partial covers are supported. Both deficits are read by `cover` alone, as its cap. No settlement function consults them.

A withdrawal while the live price sits below the ratchet. `_accrueGrowth` returns early on a dip without advancing `lastPrice`, yet `take` consumes tokens priced at the live price while reducing `withdrawBase` by the base amount. More tokens leave than the ratchet implies, with no admin action involved.

With `depositBase` and `depositProUSD` both at 1000 and a price of 1.0, an admin markdown to 0.8 records a 250 deficit and leaves the pool holding 1000 against a 1250 requirement. If cover is skipped and the price returns to 1.0, the next settlement subtracts 1000 from 1250 and banks 250 as growth, though the true excess is zero. The pool is left holding 750 against an obligation needing 1000.

Reaching that state requires a genuine loss event, an admin markdown, a skipped or partial cover, a price recovery, and a `claimGrowth` call.

Impact

Backing owed to depositors is banked as protocol yield and paid out, and the last proUSD+ holder to exit cannot withdraw.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L148-L159 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L239-L240 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L474-L491

Tool Used

Manual Review

Recommendation

Bank each pool's excess over its current requirement rather than the change in that requirement, in both branches of `_accrueGrowth` and in the `claimableGrowth` view. The two are identical while a pool is exactly backed, but the excess form yields zero while a shortfall is outstanding, so a price recovery retires the deficit instead of banking it. The existing size clamps then become redundant.

The deficit assignment in `markdownPrice` needs no change, since the shortfall is already recomputed absolutely from current state.

As defence in depth, refuse to pay `claimGrowth` while either deficit is non-zero.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/287d57d
- Banking now considers current requirement instead of change

0xklapouchy

Fix confirmed in 287d57d.

Issue L-1: `_executeUnlockedMerge` folds locked rewards into principal, inflating `totalDepositsBase` past `depositCap` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/28

Summary

When merging unlocked positions, the new position is created with `amount` set to the sum of principal **and** rewards, and `lockedRewards` set to 0. The deactivation loop only subtracts the principal of the source positions, so `totalDepositsBase` grows by the merged rewards. `depositCap` is enforced only in `_executeDeposit`, so the merge is never checked against it and the cap can be exceeded without any new capital entering the protocol.

Vulnerability Detail

The three relevant statements in `_executeUnlockedMerge`:

```
totalAmount += position.amount + position.lockedRewards;   // L550, principal and
↪  rewards combined
totalDepositsBase -= positions[positionIds[i]].amount;     // L555, principal only
↪  removed
totalDepositsBase += totalAmount;                          // L576, principal and
↪  rewards added back
```

The net movement on `totalDepositsBase` is therefore `+ sum(lockedRewards)`. The new position stores that combined figure as `amount` with `lockedRewards: 0`, so the reward component permanently loses its identity and is thereafter treated as principal.

`depositCap` is checked at a single site, `_executeDeposit` line 183. `unlockedMerge` is a direct call requiring no authority proof, so any position owner can push `totalDepositsBase` above the cap at will. Once inflated, the cap also blocks legitimate new deposits earlier than intended.

Impact

`depositCap` can be exceeded without new capital, and rewards are reclassified as principal.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L550-L576 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L183

Tool Used

Manual Review

Recommendation

Accumulate principal and rewards separately and carry both onto the merged position, rather than collapsing them into `amount`:

```
totalAmount  += position.amount;
totalRewards += position.lockedRewards;
...
amount: totalAmount,
lockedRewards: totalRewards
```

With `totalDepositsBase += totalAmount` then adding back exactly what the deactivation loop removed, the net movement is zero and the cap is preserved without needing a re-check. Rewards remain tracked as rewards, and the withdrawal path is unchanged because it already sums both fields.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/4516002
- _executeUnlockedMerge now separately accumulates position.amount and position.lockedRewards
- https://github.com/promis-finance/promis-contracts/commit/2f9c108
- Removed unnecessary subtraction and addition to totalDepositsBase

0xklapouchy

Fix confirmed in 4516002 + 2f9c108.

Issue L-2: `unlockedPositionsToMerge` is bound into the proof and never cross-checked, so a withdraw request containing an overlapping id can never be executed [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/29

Summary

`unlockedPositionsToMerge` is stored at request creation and hashed into the EIP-712 struct, so the list is frozen for the life of the request. The merge it triggers is a synchronous, self-scoped operation that needs no backend approval, yet it is executed inside the proof-gated finalize path and runs before the host function's own validation. In `executeCreateWithdrawRequest` the merge ids are never checked against `_positionIDs`, so a request created with an id present in both arrays can never be finalized.

Vulnerability Detail

`_detectDuplicatedPositionIds` is applied to `_positionIDs` alone at line 211. The two arrays are never compared against each other, at creation or at finalize.

At finalize, `_executeInitiateWithdraw` merges first and validates afterwards:

```
_mergeUnlockedIfProvided(caller, unlockedPositionsToMerge);   // L285, deactivates
↪  the merged ids
...
if (position.status != ProTokenPlusTypes.PositionStatus.ACTIVE) {
    revert IProTokenPlus.PositionNotActive(posId);            // L298, the withdraw
    ↪  loop then rejects them
}
```

If an id appears in both arrays, the merge sets it to `UNLOCKED_MERGED` and the withdraw loop immediately reverts `PositionNotActive`. The merge requires two or more entries to run, so the trigger is any withdraw request where `unlockedPositionsToMerge` holds at least two ids and one of them is also in `_positionIDs`.

Both arrays are committed to `WITHDRAW_PROOF_TYPEHASH` at line 263, so the authority cannot reissue a corrected proof; only `_proofKind` and `_deadline` are free. Line 255 rejects every proof kind except `PROOF_OF_APPROVE`, so there is no return or void path either. The request is stuck in `PENDING` permanently.

The same coupling creates a second failure mode without any overlap. The merge ids are neither validated nor escrowed at creation, so consuming them in the interval via `unlockedMerge`, `relock` or another request makes the frozen list stale and the finalize reverts the same way.

No funds are escrowed by a withdraw request and the positions stay `ACTIVE`, so the user can abandon the request and create a new one.

Impact

A withdraw request can be created in a state where it can never be finalized, and the merge list frozen in the proof cannot be corrected.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L204-L239 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L279-L307

Tool Used

Manual Review

Recommendation

Remove `unlockedPositionsToMerge` from the request structs and from `DEPOSIT_PROOF_TYPEHASH` and `WITHDRAW_PROOF_TYPEHASH`. Merging is a user-scoped action on the caller's own positions that requires no authority attestation, and binding it into the signed digest only adds a failure mode: the list cannot be corrected once signed, and the authority is made to attest to state it does not control.

If the combined call is kept for user convenience, execute the merge at request creation, where the ids arrive as fresh calldata and can be validated immediately, rather than at finalize. Keep it independent of the host operation so a merge outcome cannot block the main action, and cross-check the merge ids against any other id array the same call consumes.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/8e40e7a
- _mergeUnlockedIfProvided removed. Merging unlocked position is now exclusive function

0xklapouchy

Fix confirmed in 8e40e7a.

Issue L-3: Redundant `pragma abicoder v2` declared in nine contracts and interfaces [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/30

Summary

Nine files declare `pragma abicoder v2` while also pinning `pragma solidity 0.8.29`. ABI coder v2 has been the default since Solidity 0.8.0, so the directive has no effect and is dead configuration.

Vulnerability Detail

Every file below declares `pragma abicoder v2;` on line 3, directly under `pragma solidity 0.8.29;` on line 2:

- `core/interfaces/IProToken.sol`
- `core/interfaces/IProTokenOperations.sol`
- `core/interfaces/IProTokenSettings.sol`
- `core/interfaces/IProTokenUnmintHandler.sol`
- `oracles/interfaces/IOracleAdaptor.sol`
- `vaults/ProTokenPlus/ProTokenPlus.sol`
- `vaults/ProTokenPlus/ProTokenPlusOperations.sol`
- `vaults/ProTokenPlus/state/ProTokenPlusState.sol`
- `vaults/ProTokenPlus/interfaces/IProTokenPlus.sol`

The declaration is also applied inconsistently: the remaining in-scope files use the same compiler version without it, and compile identically.

Impact

Dead configuration; no security impact.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlus.sol#L2-L3

Tool Used

Manual Review

Recommendation

Remove the `pragma abicoder v2` line from all nine files.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/8e40e7a
- abicode v2 pragma removed

0xklapouchy

Fix confirmed in 8e40e7a.

Issue L-4: The `amount == 0` full-balance sentinel is unreachable [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/31

Summary

`IYieldProtocolHandler.withdrawYieldAsset` documents `amount == 0` as a request for the full balance, and both `AaveV3YieldHandler` and `MorphoYieldHandler` implement that branch. `YAssetOperationsHandler` is the only authorized caller of either handler, and it rejects a zero amount at every call site. The documented sentinel can therefore never be exercised, and the branch is dead code in both handlers.

Vulnerability Detail

The interface states the convention twice, on the function and on the event:

```
* @param amount Requested amount to withdraw (asset decimals). Supply 0 to request
 ↪  the full balance.
* @param amount Requested amount to withdraw (asset decimals). A value of 0
 ↪  indicates
*         the caller requested the full balance.
```

Both handlers honour it. `AaveV3YieldHandler` and `MorphoYieldHandler` resolve a zero amount to `this.getBalance()`.

Both also restrict the caller with `if (msg.sender != operationsContract) revert Unauthorized();`, so `YAssetOperationsHandler` is the only address that can ever reach the function. Every path there excludes zero:

- `withdrawalYieldAssets` reverts `ZeroAmount` on entry
- `withdrawalYieldAssetsMultiple` reverts `ZeroAmount` per item
- `payOut` computes `toPull` inside a loop guarded by `remaining > 0` and skips any handler with `handlerBalance == 0`, so `toPull` is always positive

The practical consequence is that the operator can only withdraw an exact amount, which must be read from `getBalance()` beforehand. Because the Aave liquidity index and the Morpho supply position accrue between the read and the transaction, the figure is stale on arrival and a residual balance is always left behind. There is no call that clears a handler completely.

That residue matters for reconfiguration. `setYieldAsset` and `setAavePool` guard on the aToken balance being zero, and `setMorphoMarketParams` and `setMorphoCoreContract` guard on `getBalance()` being zero, so leftover dust blocks the very operations the guards are written for.

The event is also affected: `YieldAssetWithdrawn` documents `amount == 0` as meaning a full-balance request, a value it can never carry.

Impact

A yield handler cannot be drained to zero in a single call.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/yields/interfaces/IYieldProtocolHandler.sol#L82-L90 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/YAssetOperationsHandler.sol#L115-L157

Tool Used

Manual Review

Recommendation

Allow zero through in `withdrawalYieldAssets` and `withdrawalYieldAssetsMultiple` when a specific handler is targeted.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/66b6856
- Zero amount for specific handler is now allowed in withdrawalYieldAssets and withdrawalYieldAssetsMultiple

0xklapouchy

Fix confirmed in 66b6856.

Issue L-5: Manually supplied `yAsset` decimals can misprice mint and unmint operations [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/32

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`setYAsset()` stores the admin-supplied `decimals` value without checking it against the yAsset. This value is later used to calculate mint and unmint amounts, so an incorrect configuration can significantly misprice both operations.

Vulnerability Detail

The protocol uses the configured decimals as the token's accounting scale. Mint calculations divide the deposited raw amount by `10 ** decimals`, while unmint calculations multiply the payout by the same value.

For example, configuring an 18-decimal token as 6 decimals makes one whole token appear `1e12` times more valuable during minting. The opposite mismatch can calculate an excessively large redemption amount. Cross-asset redemptions can then expose correctly configured backing to the incorrect accounting.

Impact

An admin configuration mistake can cause excessive proUSD minting, excessive yAsset payouts or queued liabilities, user underpayment, or failed mint/unmint operations.

Code Snippet

`contracts/core/ProTokenSettings.sol` stores the supplied value directly:

```
storageSettings.decimals = _settings.decimals;
```

`contracts/core/ProTokenOperations.sol` uses it in the conversion calculations:

```
uint256 cappedYAssetAmountUSD = (yAssetAmount *
    cappedYAssetUSD) / (10 ** yAssetDecimals);

uint256 oneYAssetUnit = 10 ** yAssetSettings.settings.decimals;
```

Tool Used

Manual Review

Recommendation

For supported metadata-compatible yAssets, read and store the decimals directly from the token during onboarding:

```
storageSettings.decimals = IERC20Metadata(_yAsset).decimals();
```

Discussion

promisque

- Acknowledged
- setYAsset() is non-frequently called admin-controlled function
- We will make sure a wrong decimal is not passed along intended asset

0xklapouchy

Acknowledged.

Issue L-6: `removeYAsset()` treats a failed balance query as zero [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/33

Summary

`removeYAsset()` should remove a yAsset only when its handler balance is zero. However, if `getYAssetInfo()` reverts, the function treats the balance as zero and deletes the yAsset from the registry even when the handler still holds funds.

Vulnerability Detail

The original is technically correct. `getYAssetInfo()` does not remove the yAsset, so adding “before removing the yAsset” there could be misleading.

Clearer wording:

Before removing a yAsset, `removeYAsset()` calls `YAssetOperationsHandler.getYAssetInfo()`. If any configured handler’s `getBalance()` call reverts, the entire balance query reverts. `removeYAsset()` catches this failure and incorrectly treats the balance as zero.

`ProTokenSettings.removeYAsset()` catches that failure but leaves `totalAmount` at its default value of zero. The zero-balance check therefore passes, after which the yAsset is removed and its settings are deleted. As a result, a failed balance query is incorrectly treated as proof that no backing exists.

The old handler and its funds remain unchanged. Once the failing read is repaired or reconfigured, registering the same yAsset and handler can be restores the registry entry and access to the backing.

Impact

A funded yAsset can be removed from the active registry, causing its backing to be temporarily omitted from registry-based monitoring and normal protocol routing. Since only the trusted admin can perform the removal and the funds remain recoverable, there is no direct theft or permanent loss. Therefore, the issue is Informational severity.

Code Snippet

In `contracts/core/ProTokenSettings.sol`, a failed query leaves `totalAmount` equal to zero:

```
address yOpsHandler = yAssetSettings[_yAsset].yOperationsHandler;
uint256 totalAmount = 0;
try IYAssetOperationsHandler(yOpsHandler).getYAssetInfo() returns (
```

```
    address,
    uint256 amount
) {
    totalAmount = amount;
} catch {
    // If call fails, totalAmount remains 0 - safe to remove misconfigured yAsset
}
if (totalAmount > 0) revert YOperationsHandlerInUseBalanceNotZero();

yAssets.remove(_yAsset);
delete yAssetSettings[_yAsset];
```

In `contracts/core/YAssetOperationsHandler.sol`, one reverting handler prevents all balances from being returned:

```
for (uint256 i = 0; i < len; ) {
    uint256 handlerBalance =
        IYieldProtocolHandler(protocolHandlers[i].handlerContract).getBalance();
    totalAmount += handlerBalance;
    unchecked { ++i; }
}

totalAmount += IERC20(yAsset).balanceOf(address(this));
```

Tool Used

Manual Review

Recommendation

Make the normal removal check fail closed by reverting when `getYAssetInfo()` fails. If removal of a broken handler is necessary, use a separate admin-only emergency removal process that requires the protocol to be paused and records the detached handler so its funds can be recovered safely.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/21951f2
- Added *balanceVerified* flag to mark a failed call. A corresponding *YAssetRemovedUnverified* event is emitted to mark this case off-chain.

Parth0x108

Partially fixed / risk accepted. The new flag and event resolve the lack of observability, but removal still proceeds when the handler balance cannot be verified, so the original fail-open behavior remains. Given the severity and explicit admin-controlled policy, I accept the remediation, with the event NatSpec corrected from “positive balance” to “unverified balance.”

promisque

- https://github.com/promis-finance/promis-contracts/commit/73d32f89de52b5025cebcc1fcd17f2504a28b82f
- Corrected NatSpec to ”unverified balance”
- Added an additional ”_forced” flag to removeYAsset(). A false ”_forced” value will result in fail closed behaviour

Parth0x108

The normal path now fails closed when the balance is unverifiable, while the explicit admin-only forced path emits the appropriate event. We consider this fully fixed.

Issue L-7: Optional Aave incentive rewards cannot be claimed through `AaveV3YieldHandler` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/34

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`AaveV3YieldHandler` stores an Aave incentives controller but does not expose any function that calls it. If the configured aToken has an active incentive program, rewards accrue to the handler but cannot be claimed through the current Promis implementation.

Normal Aave lending interest is unaffected because it is already reflected in the handler's increasing aToken balance. This issue concerns only optional, separately distributed incentive rewards.

Vulnerability Detail

Promis supplies assets to Aave on behalf of `address(this)`, making the handler the aToken holder and reward beneficiary. Aave's ordinary `claimAllRewards()` function claims rewards for its caller, so the handler must call the Rewards Controller itself unless Aave separately authorizes an on-behalf claimer.

The handler declares the claim interface and lets the admin set `incentivesController`, but the setter explicitly stores the address for future use and no function invokes `claimAllRewards()`. The existing `emergencyWithdraw()` also cannot help because it only transfers tokens already held by the handler; it cannot claim rewards still recorded in Aave.

Impact

If in case incentives are enabled for the configured aToken, optional reward tokens can remain unclaimed until the handler is upgraded or an authorized on-behalf claimer is configured. This is delayed or unrealized optional strategy revenue rather than principal loss, theft, or an irreversible loss of funds. No active incentives configuration or accrued amount is established by the repository, so the issue is Informational.

Code Snippet

In `contracts/yields/AaveV3YieldHandler.sol`, deposits are made on behalf of the handler:

```
IERC20(asset).forceApprove(pool, amount);
IPool(pool).supply(asset, amount, address(this), 0);
```

However, the incentives controller is only stored for future use:

```
/**
 * @notice Sets the AAVE incentives controller address
 * @dev Only callable by admin. The incentivesController is stored for future use
 *      to enable claiming AAVE rewards when the feature is implemented.
 */
function setIncentivesController(
    address _incentivesController
) external onlyAdmin {
    incentivesController = _incentivesController;
    emit IncentivesControllerUpdated(_incentivesController);
}
```

The reported balance includes only the aToken balance:

```
function getBalance() external view returns (uint256) {
    address aTokenAddress = _getATokenAddress();
    if (aTokenAddress == address(0)) {
        return 0;
    }
    return IAToken(aTokenAddress).balanceOf(address(this));
}
```

Tool Used

Manual Review

Recommendation

If Aave incentives are supported, add an admin-only, non-reentrant claim function that calls the aToken's actual Rewards Controller with the aToken in the assets array. Send rewards to a fixed protocol-controlled recipient, emit the claimed token addresses and amounts, and define whether the rewards belong to the treasury or should be converted into the configured yAsset and added to protocol backing.

Discussion

promisque

- Acknowledged

- If any incentives are supported later on, we will add the functionality via upgrade

0xklapouchy

Acknowledged.

Issue L-8: `setYProtocolHandlers()` can detach funded handlers from accounting and payouts [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/35

Summary

`setYProtocolHandlers()` replaces the existing handler list without checking whether a removed handler still holds funds. The funds remain in the old handler, but it is excluded from aggregate balance reporting, payout routing, and normal withdrawals until the admin re-adds it.

Vulnerability Detail

When the admin replaces the handler list, the function clears every old handler from `isProtocolHandler` and deletes the complete `protocolHandlers` array. It does not first check the old handlers' balances or withdraw their funds.

After removal, `getYAssetInfo()`, `previewPayOut()`, and `payOut()` only inspect the new handler list. `withdrawalYieldAssets()` also rejects the old handler because its membership flag is false. Therefore, backing held by the old handler becomes temporarily unavailable through the normal routing functions.

The handler contract and its funds are not deleted. Re-adding the old handler restores balance reporting and allows the funds to be withdrawn.

Impact

The protocol can temporarily underreport its backing and queue or fail payouts even though sufficient funds remain in a removed handler. No funds are permanently lost, and only the trusted admin can create and recover this state. Therefore, the issue is Low/info severity.

Code Snippet

In `contracts/core/YAssetOperationsHandler.sol`, the old handlers are removed without a balance check:

```
uint256 oldLen = protocolHandlers.length;
for (uint256 i = 0; i < oldLen; ) {
    address old = protocolHandlers[i].handlerContract;
    if (old != address(0)) isProtocolHandler[old] = false;
    unchecked { ++i; }
```

```
}
delete protocolHandlers;
```

Normal withdrawals then reject a removed handler:

```
if (!isProtocolHandler[_handler]) revert ProtocolHandlerNotFound();

actualAmount = IYieldProtocolHandler(_handler).withdrawYieldAsset(
    _amount
);
```

Tool Used

Manual Review

Recommendation

Before removing an old handler, require a successful `getBalance()` call to return zero. For funded handlers, withdraw or migrate the complete balance while the handler is still registered, then replace the handler list. Any failed balance query should also make the replacement revert.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/a0e623f
- Removing old handler now checks if call failed via *verified* flag. A corresponding event is emitted for this case for the backend.

Parth0x108

The new verification flag and event improve observability, but the issue is only partially fixed because funded or unreadable handlers can still be detached from accounting and payout routing. The event should also be emitted only for handlers actually absent from the new configuration, otherwise allocation-only updates can produce false removal alerts.

promisque

- https://github.com/promis-finance/promis-contracts/commit/55c8c894f29a6eb7fd9f37593f70cb033c53e2cf
- Event is only emitted when old handler is not present in the new list, which means removal. Allocation only updates no longer emit event.
- Added an additional "_forced" flag to setYProtocolHandlers(). A false "_forced" value will result in fail closed behaviour

Parth0x108

The revised list-difference, event, and forced-removal logic is correct.

The issue is solved from Promis team, so just note that: the outer verified flag treats any successful zero return as authoritative, while Aave can internally convert failed aToken discovery into zero despite holding a funded position.

Issue L-9: A reverting handler balance query blocks the queued-unmint fallback [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/36

Summary

`previewPayOut()` directly calls `getBalance()` on each configured yield handler. If one handler reverts, unmint finalization also reverts before the protocol can attempt `payOut()` or create a queued unmint request.

Vulnerability Detail

`ProTokenOperations` calls `previewPayOut()` outside the `try/catch` used for `payOut()`. Therefore, when the direct reserve cannot cover the payout and an inspected handler becomes unreadable, its revert stops the entire flow.

This behavior is inconsistent with `payOut()`, which already catches failed balance reads and skips unreadable handlers. The same issue also affects `strategicUnmint()`.

Impact

An approved unmint cannot be finalized or moved to the queue while the handler remains unreadable. The transaction rolls back completely, so the request stays pending and the user's escrowed proUSD remains safe. The user can retry after recovery or receive an authority-signed return proof, making this a temporary and recoverable Low-severity liveness issue.

Code Snippet

In `contracts/core/YAssetOperationsHandler.sol`, the handler call is not protected:

```
for (uint256 i = 0; i < len; ) {
    address h = protocolHandlers[i].handlerContract;
    if (h != address(0)) {
        available += IYieldProtocolHandler(h).getBalance();
        if (available >= amount) return true;
    }
    unchecked { ++i; }
}
```

In `contracts/core/ProTokenOperations.sol`, the preview executes before the protected payout and queue fallback:

```
if (backlog == 0 && yOps.previewPayOut(yAssetToReceiveAmount)) {
    try yOps.payOut(_recipient, yAssetToReceiveAmount) {
        paidInstant = true;
    } catch {
        // fall through to the queue path below
    }
}
```

Tool Used

Manual Review

Recommendation

Make `previewPayOut()` skip unreadable handlers, matching `payOut()`:

```
if (h != address(0) && h.code.length != 0) {
    try IYieldProtocolHandler(h).getBalance() returns (uint256 balance) {
        available += balance;
        if (available >= amount) return true;
    } catch {
        // Treat an unreadable handler as zero available liquidity.
    }
}
```

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/3323d86
- Now skips unreadable handlers matching payOut()

Parth0x108

Fixed correctly. `previewPayOut()` now skips non-contract handlers and catches reverting `getBalance()` calls, matching `payOut()`'s defensive behavior. An unreadable handler therefore no longer reverts unmint finalization, when readable liquidity is insufficient, the flow correctly proceeds to queued settlement.

Issue L-10: Users can receive less value when a yAsset price differs from `usdCap` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/37

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Minting values a yAsset at `min(price, usdCap)`, while unminting values it at `max(price, usdCap)`. This pricing protects protocol funds and prevents profitable conversions between yAssets, but it can give users less value than an oracle spot-price calculation when the yAsset price differs from its configured cap.

Vulnerability Detail

If a yAsset trades at `$1.10` while its cap is `$1.00`, depositing 100 yAssets worth `$110` credits only `$100` and mints approximately 100 proUSD, assuming proUSD is worth `$1`. Redeeming those 100 proUSD into the same asset returns approximately `90.91` yAssets before fees, worth `$100` at the oracle price.

The user is not credited for `$10` of the deposited value because minting ignores the price above the cap. The redemption then correctly returns the yAsset equivalent of the user's `$100` proUSD claim. Conversely, when the yAsset trades below its cap, unminting uses the higher cap and returns fewer yAsset tokens than spot pricing would.

The formulas are explicit and always protect the protocol against unfavorable conversions. However, users may not expect to receive less value unless the cap behavior is clearly disclosed before they submit a request.

Impact

Users can unknowingly receive less value than a spot-price conversion when a yAsset deviates from its cap. There is no attacker-controlled theft or protocol loss, and `minAmountOut` allows users to reject an unacceptable quote. The issue is therefore Informational.

Code Snippet

Minting caps the credited yAsset price:

```
// Cap the yAsset USD value at the configured cap: min(price, cap)
uint256 cappedYAssetUSD = yAssetUsdRep.assetUSD >
    yAssetPriceSettings.usdCap
    ? yAssetPriceSettings.usdCap
    : yAssetUsdRep.assetUSD;
```

```
uint256 cappedYAssetAmountUSD = (yAssetAmount *
    cappedYAssetUSD) / (10 ** yAssetDecimals);
return
    (cappedYAssetAmountUSD * USD_PRECISION) / proTokenUsdRep.assetAmountUSD;
```

Unminting floors the yAsset price at the cap:

```
// Floor the yAsset USD value at the configured cap: max(price, cap).
uint256 flooredYAssetUSD = yAssetUsdRepresentation.assetUSD <
    yAssetSettings.settings.priceSettings.usdCap
    ? yAssetSettings.settings.priceSettings.usdCap
    : yAssetUsdRepresentation.assetUSD;

return
    (proTokenUsdRepresentation.assetAmountUSD * oneYAssetUnit) / flooredYAssetUSD;
```

Tool Used

Manual Review

Recommendation

Clearly document both pricing formulas and show how much value the user will receive in the UI before request submission. Warn users when the oracle price differs from `usdCap`, and display the spot value, credited or redeemed value, and fees separately.

Discussion

promisque

- Acknowledged
- The pricing formulas will be documented

0xklapouchy

Acknowledged.

Parth0x108

Acknowledged/fixed by improving documentation.

Issue L-11: `setUSDPrice` does not update `lastPriceUpdateAt` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/38

Summary

`lastPriceUpdateAt` is written only by `updateUSDPrice`. The admin path, `setUSDPrice`, changes the price without touching it, so an admin correction consumes no part of the priceOperator's cooldown. If the previous operator cooldown has already elapsed, the priceOperator can overwrite the admin's price instantly.

Vulnerability Detail

`updateUSDPrice` gates the priceOperator on `lastPriceUpdateAt + priceUpdateCooldown` and refreshes the timestamp when it succeeds. `setUSDPrice` sets `usdPrice` and emits, leaving the timestamp untouched.

This can affect the markdown procedure. The admin lowers the price ahead of running `StrategyVault.markdownPrice()` and `cover()`. Because the stale timestamp leaves the cooldown already satisfied, the priceOperator can raise the price back instantly. That both undoes the correction and blocks the remediation, since `markdownPrice()` requires the live price to be below `lastPrice` and otherwise reverts `NotAMarkdown`.

The `initialize` also leaves `lastPriceUpdateAt` at zero, so the first operator update is ungated for the same reason.

Impact

An admin price correction can be immediately overwritten by the priceOperator.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProToken.sol#L119-L126 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProToken.sol#L129-L147

Tool Used

Manual Review

Recommendation

Have `setUSDPrice` write `lastPriceUpdateAt = block.timestamp`. The admin path should not be gated by the cooldown, since it must remain usable in an emergency, but it should reset it so an admin action is guaranteed a full cooldown window before the priceOperator can act. Seed the same field in `initialize` so the first operator update is gated as well.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/7f6f9df
- setUSDPrice does update lastPriceUpdateAt
- initializer now also updates lastPriceUpdateAt

0xklapouchy

Fix confirmed in 7f6f9df.

Issue L-12: Initializers set state without emitting the corresponding setter events [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/39

Summary

Every field written by an initializer has a dedicated setter that emits an event, but no initializer emits anything. Consumers that reconstruct state from logs therefore see every later change to these values and never their genesis value.

Vulnerability Detail

None of the eight initializers in scope contain a single `emit`, while each of the fields they write has an event-emitting setter:

- `ProToken`: `minter`, `usdPrice`, `priceUpdateCooldown` (setters emit `MinterSet`, `USDPriceSet`, `PriceUpdateCooldownChanged`)
- `ProTokenSettings`: `admin`, `operator`, `priceOperator` (`AdminAccepted`, `OperatorSet`, `PriceOperatorSet`)
- `ProTokenOperations`: `minDepositBase`, `minWithdrawBase` (`MinDepositBaseSet`, `MinWithdrawBaseSet`)
- `ProTokenUnmintHandler`: `unmintBatchDuration` (`UnmintBatchDurationUpdated`)
- `ProTokenPlus`: `proUSD`, `unbondingPeriod`, and the initial tiers (`ProUSDSet`, `UnbondingPeriodSet`, `TierAdded`)
- `StrategyVault`: `proTokenPlus`, `proTokenOperations` (`ProTokenPlusSet`, `ProTokenOperationsSet`)
- `AaveV3YieldHandler`: `operationsContract`, `aToken` (`OperationsContractUpdated`, `ATokenUpdated`)
- `MorphoYieldHandler`: `operationsContract`, `morphoCoreContract`, `morphoMarketParams` (`OperationsContractUpdated`, `SetMorphoCoreContract`, `MorphoMarketParamsUpdated`)

Impact

Log-only consumers cannot reconstruct the genesis configuration.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProToken.sol#L6

6-L83

Tool Used

Manual Review

Recommendation

Emit the matching event for each value assigned in an initializer, using the zero value or zero address as the previous value where the event carries one.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/f33d3bd
- initializers now emit corresponding update events

0xklapouchy

Fix confirmed in f33d3bd.

Issue L-13: Passive contract receivers cannot claim queued redemptions [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/43

Summary

An instant unmint can send yAssets to any receiver, but a queued unmint requires the receiver itself to call `ProTokenUnmintHandler::claimUnmintRequests()`. A passive contract that can receive tokens but cannot make external calls will therefore be unable to claim.

Vulnerability Detail

When liquidity is unavailable, the chosen recipient is stored as the queue receiver. Claiming later requires `msg.sender` to equal that receiver, and there is no function allowing the original user or another account to trigger payment to the stored receiver.

Impact

The yAssets of a user-selected passive contract receiver can remain stuck in `ProTokenUnmintHandler`.

Code Snippet

`contracts/core/ProTokenOperations.sol` stores the recipient in the queued request:

```
IProTokenUnmintHandler(
    proTokenInfo.proTokenUnmintHandler
).createUnmintRequest(_recipient, _yAsset, yAssetToReceiveAmount);
```

`contracts/core/ProTokenUnmintHandler.sol` requires that receiver to call and pays `msg.sender`:

```
if (unmintRequest.receiver != sender) revert Unauthorized();

IERC20(yAsset).safeTransfer(sender, totalTransferAmount);
```

Tool Used

Manual Review

Recommendation

Allow the original requester or any account to trigger the claim, but always transfer the yAssets to the stored receiver.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/f2a8526
- Anybody can now trigger claim and assets go to stored receiver

0xklapouchy

Fix confirmed in f2a8526.

Issue L-14: Unreachable per-price loop in `_validatePriceDeviation` wastes gas on every deviation check [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/45

Summary

`_validatePriceDeviation` first rejects the spread between the lowest and highest price, then loops every price comparing it against the median. The loop can never revert, because passing the spread check mathematically guarantees every individual deviation is within the same bound. It is dead code that costs gas per oracle.

Vulnerability Detail

The array is sorted before this function runs, so `min <= median <= max`. For any price in the array, `|price - median| <= max - min`, and since `median >= min`:

```
deviation = |price - median| / median  <=  (max - min) / median  <=  (max - min) /
↪  min  =  spread
```

Floor division is monotonic, so the ordering also holds for the integer results the contract computes. If `spread <= maxPriceDeviation` passes, then every per-price deviation is at most `maxPriceDeviation` as well, and the loop's revert condition is unsatisfiable.

Impact

Wasted gas per oracle on every deviation check; no security impact.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L916-L941

Tool Used

Manual Review

Recommendation

Delete the per-price loop and drop the now-unused `medianPrice` parameter from the signature and its call site. The spread check alone is sufficient and strictly cheaper.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/d8c534d
- Deleted per-price loop and dropped medianPrice parameter

0xklapouchy

Fix confirmed in d8c534d.

Issue L-15: `setProToken` cannot achieve a migration and every use of it desynchronizes StrategyVault [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/46

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`ProTokenSettings.setProToken` allows the proUSD address to be replaced at any time, but the change cannot be propagated. `StrategyVault` caches `proToken` at initialize and exposes no setter for it, so it can never follow the new address without a contract upgrade. Every reachable use of the setter therefore leaves the system inconsistent rather than migrated.

Vulnerability Detail

`ProTokenOperations` reads `proToken` from settings on every call, so it switches to the new address immediately. `StrategyVault` does not. It has setters for `proTokenPlus`, `proTokenOperations` and `yieldRecipient`, but none for `proToken`, and `ProTokenPlus` tracks its own `proUSD` behind a separate setter.

After a change, the vault still holds and accounts the old token while Operations transacts in the new one:

- `borrow` calls `strategicUnmint`, which burns the new token from the vault. The vault holds only the old token, so it reverts and borrowing is bricked.
- `repay` mints the new token into the vault while `take` transfers the old one, so the reserve ledger and actual holdings diverge.
- Existing holders cannot redeem, because `createUnmintRequest` calls `safeTransferFrom` on the new token, which they do not hold.

The mint path continues to work against the new token, so there is no loud failure. The protocol appears operational while prior holders are silently stranded.

The usual justification for a mutable token address, replacing a defective token, does not apply. `ProToken` is itself UUPS upgradeable with admin-gated `_authorizeUpgrade`, so a token defect is fixed in place by upgrading the implementation. Address migration is never required.

This distinguishes `proToken` from its siblings. `setProTokenOperations` and `setProTokenUnmintHandler` point at logic contracts that hold no user balances and are legitimately replaceable. `proToken` holds every user's balance and represents the protocol's liability.

Impact

The setter has no valid use, and any use of it breaks vault accounting and strands existing holders.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenSettings.sol#L112-L118 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L57-L81

Tool Used

Manual Review

Recommendation

Remove `setProToken` and assign the address once in `initialize`, relying on the token's own upgradeability for any future fix. If a migration path must remain, add a matching setter to `StrategyVault` so the change can actually be propagated, and gate the whole rotation behind a single atomic operation that updates every consumer together.

Discussion

promisque

- Acknowledged
- The setter is only meant to be used on first deployment since both ProTokenSettings and ProToken require each other addresses
- The ProToken address is very highly unlikely to be changed

0xklapouchy

Acknowledged.

Issue L-16: `minDepositBase` is checked against the uncapped USD value while the mint credits the capped one [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/47

Summary

`_validateMintInputs` evaluates the deposit floor using the raw price returned by `_getUsdRepresentationYAsset`, which applies no cap. Every mint path that actually credits proUSD values the same deposit at `min(price, usdCap)`. The gate and the accounting therefore use different valuations of the same input.

Vulnerability Detail

`_getUsdRepresentationYAsset` contains no reference to `usdCap`. It returns the static or median oracle price as-is. The cap is applied only inside `_calculateMintingAmount`, which clamps to `min(price, usdCap)` before deriving the mint amount, and inside `_calculateUnmintAmount`, which floors at `max(price, usdCap)`.

The two diverge only when the asset trades above its cap, since below it `min(price, usdCap)` is the price itself. In that case the floor is evaluated at a higher valuation than the protocol is willing to credit, so it is under-enforced by `(price - usdCap) / price`. On the default floor of 100e18 with an asset one or two percent above cap, that is one or two dollars.

The contract header states these floors exist so that "the minDepositBase/minWithdrawBase floors prevent dust exploitation", in the context of the divide-before-multiply precision loss described there. That precision loss occurs in the capped arithmetic, so the floor intended to bound it should be measured against the same quantity.

The unmint side has no equivalent divergence. `minWithdrawBase` is evaluated through `_getUsdRepresentationProToken`, and proUSD has no cap, so gate and accounting agree by construction.

Impact

The deposit floor is slightly under-enforced whenever a yAsset trades above its configured cap.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L610-L634 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L786-L810

Tool Used

Manual Review

Recommendation

Apply the same `min(price, usdCap)` clamp before comparing against `minDepositBase`, so the floor is measured against the value the protocol actually credits.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/139994f
- minDepositBase is now checked against capped price

0xklapouchy

Fix confirmed in 139994f.

Issue L-17: `ProTokenMint` emits a price that was not used, contradicting its own documentation [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/48

Summary

The `ProTokenMint` event documents itself as "a complete record of the minting transaction including exchange rates used", but `ctx.yAssetUSD` is assigned the raw oracle or static price rather than the capped price the mint actually applied. The rate that was applied cannot be recovered from logs either, because `usdCap` is never emitted by any event.

Vulnerability Detail

`_calculateMintAmount` computes the mint amount through `_calculateMintingAmount`, which clamps the yAsset valuation to `min(price, usdCap)`. It then assigns the context field from the unclamped representation, and `_executeMint` forwards that value into the event.

The interface states:

```
@dev This event provides a complete record of the minting transaction including
↪  exchange rates used.
@param yAssetUSD The USD price per unit of the y asset at time of minting, in 18
↪  decimals.
```

The emitted figure is therefore not the rate used. A consumer checking the natural relation between the emitted fields, `proTokenAmount` against `yAssetAmount * yAssetUSD / proTokenUSD`, gets a wrong answer whenever the asset trades above its cap. That is precisely the condition the cap exists to handle, so the event is unreconcilable exactly when it matters most. Below the cap the clamp is a no-op and the fields agree.

The applied rate is also not derivable from event data. `setYAsset` emits only `YAssetSet(address indexed yAsset)` with no settings payload, so no log ever carries `usdCap`. Since that value is mutable through the same function, determining what the cap was at the block of a past mint requires archive-node `getYAssets` calls rather than log replay. A consumer cannot apply the clamp themselves after the fact.

Impact

Off-chain consumers cannot reconcile or reconstruct the exchange rate actually applied to a mint.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L663-L671 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L683-L691

Tool Used

Manual Review

Recommendation

Assign the capped value to `ctx.yAssetUSD` so the event matches the documented meaning and the emitted fields reconcile. If the raw oracle reading is wanted as an audit trail, emit both the observed price and the applied price and document the difference.

Separately, give `YAssetSet` a settings payload so `usdCap` changes are reconstructible from logs, since without it no consumer can verify the clamp independently.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/203fef3
- ctx.yAssetUSD is now assigned capped price

0xklapouchy

Fix confirmed in 203fef3.

Issue L-18: A recipient-side payout failure is treated as a liquidity shortfall, letting any user suspend instant redemption for a whole yAsset [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/49

Summary

`_executeUnmint` catches every `payOut` revert and routes the request to the queue, incrementing the per-asset backlog. Instant redemption is then gated on that backlog being zero until an operator processes a batch. The catch cannot distinguish a genuine liquidity shortfall from a failure caused by the recipient, so a user who nominates a blocklisted address forces a queue entry while reserves are fully sufficient, suspending instant redemption for everyone using that asset.

Vulnerability Detail

The gate is correct when the protocol cannot pay. Queueing and operator-funded batches are the designed response to insufficient liquidity, and blocking later redemptions preserves FIFO fairness. The defect is that the same path fires when the protocol **can** pay and only the delivery failed.

`payOut` short-circuits to `IERC20(yAsset).safeTransfer(to, amount)` whenever unallocated balance covers the request, and that transfer reverts for an address on a token blocklist. USDC and USDT both carry one. The recipient is user-supplied at request creation and defaults to the caller only when zero, so nominating a known blocklisted address is sufficient. The `catch` swallows the revert, the request queues, and `unpaidQueuedLiability` rises.

That counter decrements only in `processNextUnmintBatch`, which reverts until the batch window has elapsed. Every subsequent unmint of that asset therefore takes the delayed, operator-dependent path for at least a full window, no matter how much liquidity the reserve holds.

The attacker forfeits the redemption: their proUSD is burned, and the queued request is claimable only by the nominated receiver, who is blocklisted and cannot claim either, so the funds are stranded permanently. The cost is one redemption at or above `minWithdrawBase`, repeatable once per window.

One mitigation exists off-chain. The receiver is committed into `UNMINT_PROOF_TYPEHASH`, so the authority signs off on it and a backend screening recipients against token blocklists would refuse the request. That screening is not enforced on-chain, and blocklist status can change between signing and finalization inside the proof deadline.

Impact

Any user can suspend instant redemption for an entire yAsset until an operator processes the next batch, at the cost of one forfeited minimum redemption.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L406-L441

Tool Used

Manual Review

Recommendation

Do not let a delivery failure touch the shared backlog. A revert originating from the recipient says nothing about the protocol's ability to service other redemptions, so it should be recorded against that request alone. Probing the transfer separately, or checking that reserves were actually insufficient before incrementing the counter, both achieve this while leaving the liquidity path unchanged.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/e240aad
- Transfers that happen from payOut now revert with PayoutDeliveryFailed incase of blocklisted recipients

0xklapouchy

Fix confirmed in e240aad.

Issue L-19: `getUnmintRequestIdForReceiverInBatch` documents 0 as ”no request” while 0 is a valid request ID [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/51

Summary

`nextUnmintRequestIdPerYAsset` starts at zero, so the first unmint request created for each yAsset is assigned ID 0. The getter that exposes a receiver's request ID within a batch documents 0 as meaning no request exists, so the two cases are indistinguishable to any off-chain consumer.

Vulnerability Detail

The interface states the sentinel twice:

```
/// @dev Returns 0 if the receiver has no request in the specified batch
/// @return The request ID, or 0 if no request exists
```

`createUnmintRequest` assigns `newRequestId` from the counter before incrementing it, so the first request per asset legitimately holds ID 0 and the getter returns 0 for it.

On-chain nothing depends on the ambiguity. `createUnmintRequest` decides between creating and aggregating using the receiver's `EnumerableSet` membership for the batch rather than testing the mapping for zero, and `claimUnmintRequests` authorises against the stored `receiver` rather than the ID. The consequence is limited to integrations that follow the documented contract, which would treat the first request of every yAsset as absent.

Note the sibling getter is correct: `getNextUnmintRequestId` documents ”the next unmint request ID that will be assigned”, and returning 0 before any request exists is the right answer.

Impact

Off-chain consumers following the documented sentinel misread the first request of each yAsset as non-existent.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/interfaces/IProTokenUnmintHandler.sol#L114-L124 https://github.com/sherlock-audit/2026-07-promis-p

rotocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenUnmintHandler.sol#L206-L221

Tool Used

Manual Review

Recommendation

Start `nextUnmintRequestIdPerYAsset` at one, matching how batch IDs already reserve zero as their absence sentinel. Alternatively drop the sentinel language from the NatSpec and have consumers test the returned request's `receiver` field instead.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/86231af
- nextUnmintRequestIdPerYAsset now starts at 1

0xklapouchy

86231af left the old increment in place.

ProTokenUnmintHandler.sol now pre-increments and increments again, so the counter advances by 2 per request and assigned IDs run 1, 3, 5, 7. Delete `unchecked { ++nextUnmintRequestIdPerYAsset[yAsset]; }` line.

Separately, under pre-increment the counter holds the last assigned ID, so `getNextUnmintRequestId()` now contradicts its own NatSpec; return counter + 1 or redocument it, matching the getCurrentBatchId convention already in the file.

promisque

- https://github.com/promis-finance/promis-contracts/commit/c9a8eb9a75237c75daa47cbdaa7f2d5e0ecb644c
- Revamped nextUnmintRequestPerYAsset increment to post increment

0xklapouchy

Fix confirmed in c9a8eb9.

Issue L-20: Storage gaps in three state contracts break the reserved-slot convention used everywhere else [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/52

Summary

Six of the nine state contracts size their gap so that used slots plus gap equals 50. The two yield handler states and the oracle adaptor state instead declare a flat `uint256[50]` on top of their used slots, giving totals of 56, 58 and 54.

Vulnerability Detail

There is no collision risk today. Each of the three contracts is followed in its inheritance list only by interfaces and by OpenZeppelin upgradeable base contracts, and OpenZeppelin v5 places those in ERC-7201 namespaced storage, so nothing contributes sequential slots after the state contract. The gap size is therefore unobserved.

The cost is that the reserved-slot budget can no longer be reasoned about uniformly. A reviewer checking upgrade safety has to derive each contract's total separately rather than relying on the convention the other six establish.

Impact

Convention inconsistency in reserved storage.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/yields/state/AaveV3YieldHandlerState.sol#L31 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/yields/state/MorphoYieldHandlerState.sol#L26

Tool Used

Manual Review

Recommendation

Resize the three gaps to `uint256[44]`, `uint256[42]` and `uint256[46]` respectively so every state contract reserves the same 50-slot budget.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/1c9bceb
- Fixed for OracleChainlinkPushAdaptorState
- https://github.com/promis-finance/promis-contracts/commit/e0ec35c
- Fixed for AaveV3YieldHandlerState and MorphoYieldHandlerState

0xklapouchy

Fix confirmed in 1c9bceb and e0ec35c.

Issue L-21: The price step limit is never initialized and can be disabled, so the price operator inherits the admin's unbounded reach [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/55

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`stepSize` gates the only magnitude limit on `updateUSDPrice`, and the guard is skipped entirely when it is zero. The initializer never assigns it, so it is zero at deploy, and `setStepSize` accepts zero at any time afterwards. Until an admin arms it, the price operator can raise the proUSD price by an unbounded amount once per cooldown.

Vulnerability Detail

`initialize` seeds `usdPrice` and `priceUpdateCooldown` from named constants and leaves `stepSize` at its default. There is no `DEFAULT_STEP_SIZE` to match the other two, and the guard in `updateUSDPrice` reads `stepSize != 0 &&`, so an unset value disables the check rather than blocking updates.

The two-tier price model is deliberate. `setUSDPrice` is admin-only and unbounded in both directions, and exists for exceptional corrections such as reflecting a loss in Aave or Morpho. `updateUSDPrice` is the price operator's routine path and is increase-only, cooldown-limited and step-limited. The step limit is what makes the price operator the lower-trust of the two roles, and it is the property the developer cited when introducing the split as the remediation for a prior High issue.

With `stepSize` at zero that property does not hold. The price operator gains the unbounded upward reach that was reserved for the admin, and the separation stops being a security boundary. The cooldown still applies, so the exposure is one unbounded increase per interval rather than an unlimited sequence.

`setStepSize` performs no validation, so the limit can also be removed after deployment.

Impact

The price operator can raise the proUSD price without limit, which is the reach the role split was built to withhold.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProToken.sol#L6

6-L83 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProToken.sol#L141 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProToken.sol#L150-L156

Tool Used

Manual Review

Recommendation

Seed `stepSize` in the initializer from a named constant, alongside `usdPrice` and `priceUpdateCooldown`, and have `setStepSize` reject zero so the limit can never be disabled. Unbounded price changes should remain reachable only through the admin's `setUSDPrice`.

Discussion

promisque

- Acknowledged
- This is intended

0xklapouchy

Acknowledged.

Issue L-22: `claimYield` can sweep the reserve pre-funded for still-locked positions [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/56

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`claimYield` transfers proUSD out of the withdraw reserve, bounded by `withdrawBase` minus `earmarkedWithdrawBase`. The earmark is created only when a holder initiates a withdrawal, while the strategist funds the reserve ahead of unlock by design. During that window the money owed to still-locked positions carries no earmark and reads as claimable surplus.

Vulnerability Detail

Two checks guard the claim and neither protects the reserve.

The obligation sum is `depositProUSD + growthProUSD`. It omits `withdrawProUSD` entirely, so the reserve contributes nothing to the balance the vault considers spoken for.

That leaves the earmark. `earmarkedWithdrawBase` rises only inside `earmark`, whose sole caller is `_executeInitiateWithdraw`, the moment a holder presses start withdrawal. The reserve is filled well before that: `ProTokenPlusOperations` states the operating procedure as the strategist refilling the vault during the unbonding period so the balance is available by the time `completeWithdraw` is callable. So the funding arrives first and the protection arrives later.

Ten positions of 1000 USD in a twelve month tier at ten percent owe 11_000 USD at unlock. The strategist repays 11_000 ahead of the unlock date, at which point `earmarkedWithdrawBase` is zero because nobody has started withdrawing. `surplusBase` therefore equals the entire 11_000, the obligation check does not see the reserve, and an operator can claim all of it to `yieldRecipient`. When the positions unlock and holders initiate, the earmark finally rises to 11_000 against a reserve of zero, and `take` reverts `WithdrawReserveUnderfunded`.

The gap is a scope mismatch inherited from the fix that introduced the earmark. It was added as the remediation for Medium 2.6.3 in the July 2026 assessment, which concerned `regive` and `rotate` racing a matured unbonding. Those two only reclassify between pools and the tokens stay in the vault, so protecting in-flight unbondings was sufficient there. `claimYield` transfers out of the contract, and the same guard does not cover an irreversible outflow that can run throughout the prefunding window.

Impact

Reserve funding a holder's principal and promised rewards can be claimed as protocol yield at any point before that holder starts withdrawing, leaving their exit unpayable.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L450-L461 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L284-L294 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L326-L327

Tool Used

Manual Review

Recommendation

Bound the claim by the whole outstanding obligation to proUSD+ holders rather than by the subset that has already begun exiting. That requires an aggregate liability accumulator, principal plus rewards already locked in at deposit, which the protocol does not currently maintain anywhere. Track it in `ProTokenPlus` and let `claimYield` take only the reserve above it.

As an immediate hardening, add `withdrawProUSD` to the obligation sum so the first check stops treating the entire reserve as free balance. That alone does not close the finding, since the reserve is genuinely surplus to that sum while positions remain locked, but it removes the case where both guards pass at once.

Discussion

promisque

- Disputed
- Requesting downgrade from Medium to Low
- Reason: claimYield is gated to admin/operator, a trusted role – so realizing this requires the protocol's own operator to claim the reserve to yieldRecipient before locked positions are covered. This is not an external exploit but an operational sequencing concern that sits entirely within the existing trust model.

The reserve is not a static prefunded balance. Capital moves continuously in and out of the MCA via ongoing borrow/repay, and beyond the strategist repaying near a position's maturity, the protocol team can manually request funds at any time. A premature claim is therefore recoverable by re-funding the reserve before completeWithdraw is callable, within the same operating rhythm the reserve already relies on.

Because claimYield is an admin-controlled function, the trust assumption is already present: an operator able to call it prematurely is equally able to fund or refund the reserve to keep exits payable. As with the fixed-APR solvency finding, this reduces to operator discipline and off-chain funding monitoring rather than an on-chain vulnerability, which is why we assess it as Low rather than Medium.

As hardening, the operator runbook should require confirming that the reserve covers outstanding liability before any claimYield call. A hard on-chain bound (claimable = withdrawProUSD – total outstanding liability) is deferred, since maintaining the liability accumulator on-chain conflicts with the off-chain-liability model already adopted for this protocol.

0xklapouchy

Low is fair.

The recoverability point carries it: we had modelled the reserve as sitting prefunded and idle, and continuous borrow and repay plus manual funding means a premature claim can be undone before `completeWithdraw` is callable. Real harm needs two things to go wrong rather than one.

Issue L-23: `unbondingPeriod` set to zero does not release requests already unbonding [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/57

Summary

`unbondingEnd` is snapshotted when a withdrawal is initiated, so setting the global `unbondingPeriod` to zero applies only to requests created afterwards. Holders already waiting keep their original end time. Every other zero-as-disabled sentinel in this codebase takes effect immediately.

Vulnerability Detail

Snapshotting is the right default and should stay. Reading the live global in the completion check would let an admin raise `unbondingPeriod` and retroactively extend exits that users had already committed to, which is far worse than a shortening not applying.

The gap is only the zero case, and zero can only ever release. It is the value an admin sets when they mean ”unbonding is off”, typically to let people out quickly, and it is precisely the holders already in the queue that it fails to reach.

The codebase treats zero as an immediate global disable everywhere else.

Impact

An admin disabling unbonding does not release holders who are already unbonding.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L373-L375 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlus.sol#L346-L351

Tool Used

Manual Review

Recommendation

Gate the stored deadline on the global still being active:

```
if (unbondingPeriod != 0 && block.timestamp < request.unbondingEnd) revert
↪  UnbondingNotComplete(request.unbondingEnd);
```

For any non-zero period the behaviour is identical to today, so the snapshot keeps protecting users against retroactive extension. Only the disable case changes, and it can only ever let someone out sooner.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/fd714b7
- Conditional now includes Zero value check

0xklapouchy

Fix confirmed in fd714b7.

Issue L-24: `LOCKED_MERGED` and `PositionMerged` describe a merge that was never implemented [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/58

Summary

The position status enum declares `LOCKED_MERGED` and the interface declares a `PositionMerged` event, both documented as covering the merge of a still-locked position. Neither is ever used.

Vulnerability Detail

`_executeUnlockedMerge` is the sole merge path and assigns `UNLOCKED_MERGED` to every source position. No function ever assigns `LOCKED_MERGED`, and no function ever emits `PositionMerged`.

Both declarations carry NatSpec describing behaviour that does not exist. The event's parameter documentation reads "Original position that was merged (now LOCKED_MERGED)", so an integrator reading the interface would reasonably conclude that locked positions can be merged and that an event announces it. Neither is true.

This is dead surface rather than a defect in live code: the enum value costs nothing at runtime, and an event that is never emitted cannot mislead an indexer that only reacts to what arrives. The cost is to anyone reading the interface to understand what the protocol supports.

Impact

Dead enum value and event, with interface documentation describing a feature the contracts do not provide.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/types/ProTokenPlusTypes.sol#L20-L28 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/interfaces/IProTokenPlus.sol#L268-L280

Tool Used

Manual Review

Recommendation

Remove `LOCKED_MERGED`, the `PositionMerged` event and the NatSpec that describes them, unless merging locked positions is planned, in which case document it as unimplemented rather than as existing behaviour. Note that removing an enum member shifts the numeric value of every member declared after it, so this belongs in a deployment rather than an upgrade to a live proxy.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/5db71de
- Removed unused Enum and Event

0xklapouchy

Fix confirmed in 5db71de.

Issue L-25: `YAssetsAllocated` is not emitted on the permissionless allocation path [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/59

Summary

`distributeYAsset` emits `YAssetsAllocated` before allocating. `distributeUnallocatedYAsset` performs the same allocation through the same internal function and emits nothing. Allocations made through the permissionless path leave no trace.

Vulnerability Detail

Both functions end in `_allocateToHandlers`, and the emit sits in the caller rather than in the shared internal, so only one of the two paths announces what it did.

The unemitted path is the permissionless one, which is the case where an event matters most. `distributeYAsset` is reachable only from the operations contract, so its activity is already inferable from the mint flow that triggered it. `distributeUnallocatedYAsset` can be called by anyone at any time, and it moves the entire idle balance into the yield handlers. Anything reconstructing where the protocol's assets sit from event history will miss those movements entirely, and will also miss the fact that the idle balance which `previewPayOut` and `payOut` draw from first has been emptied.

Impact

Allocations performed through the permissionless path are invisible in event history, so off-chain accounting of handler balances silently diverges.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/YAssetOperationsHandler.sol#L74-L80 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/YAssetOperationsHandler.sol#L87-L112

Tool Used

Manual Review

Recommendation

Move the emit into `_allocateToHandlers` so every allocation announces itself regardless of which entry point reached it, and delete the one in `distributeYAsset` to avoid emitting twice.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/ffc7fbe
- Event is moved into a common area

0xklapouchy

Fix confirmed in ffc7fbe.

Issue L-26: A tier added without a name can never be updated [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/60

Summary

`addTier` copies the tier name from calldata without requiring it to be non-empty, while `updateTier` treats an empty stored name as proof the tier does not exist and reverts. A tier created with an empty name is fully usable for deposits, and its configuration is frozen permanently.

Vulnerability Detail

`addTier` validates only two things: that the slot is not already taken, and that a non-floor tier has a non-zero duration. The name is written straight from the supplied config.

`updateTier` opens with an existence check that uses the name as its proxy:

```
if (bytes(tier.name).length == 0) revert TierError(tierId);
```

A tier stored with an empty name therefore fails that check forever. Its APR, duration, minimum deposit, active flag and depositable flag can never be changed, and no other function writes the name, so the state is unreachable by any path.

The tier still functions. Deposits read `tiers[tierId]` directly and check `isActive` and `isDepositable`, neither of which depends on the name, so users can lock funds into a tier whose terms the admin has permanently lost the ability to correct.

The same empty-name state also defeats the duplicate guard in `addTier`, which reads `isActive || name.length > 0`. Neither is an existence flag, so a tier that is inactive and unnamed reads as absent and can be added again, pushing the same id into `tierIds` a second time. The tier-listing view iterates that array and returns the duplicate twice, and the array grows on every re-add. A non-empty name makes the guard trip regardless of the active flag, so this state is reachable only through the same omission.

Impact

A tier created with an empty name accepts deposits but its configuration can never be changed, including deactivating it.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/P

roTokenPlus.sol#L285-L300 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlus.sol#L318-L326

Tool Used

Manual Review

Recommendation

Reject an empty name in `addTier`, and in the batch tier setup the initializer performs.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/b3b9ce5
- Empty tier name is no more allowed

0xklapouchy

Fix confirmed in b3b9ce5.

Issue L-27: Deposit requests escrow proUSD before any tier is validated [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/61

Summary

`executeCreateDepositRequest` takes custody of the caller's proUSD and then validates only a minimum deposit that defaults to zero. Whether the tier exists, is active, is depositable, or leaves room under the cap is checked later, at finalization, behind an authority proof. A request that can never be approved can therefore be created, and the escrowed proUSD has no user-side exit.

Vulnerability Detail

The create path transfers the proUSD in, reads the tier config, and checks `minDeposit` alone. Solidity returns a zero-filled struct for an unset mapping key, so an id that was never configured yields `minDeposit == 0` and passes. The function does not check that the tier exists at all, let alone that it accepts deposits.

Every gate that matters sits in the finalize path instead: the tier must be active, must be depositable, and the new amount must fit under `depositCap`. All three are evaluated only when the authority approves the request, which is after the funds are already held.

The result is a request that reverts whenever anyone tries to approve it, with the user's proUSD locked behind it. Nothing lets the depositor retract. There is no cancel function and no expiry on a deposit request, so the only route out is a return proof, which is itself an authority signature. A user who mistypes a tier id, or targets one that was paused a moment earlier, cannot recover their own funds without the backend choosing to act.

The window is not limited to user error either. Tier configuration is mutable, so a request that was entirely valid when created becomes unapprovable the moment an admin deactivates that tier or lowers the cap below the pending amount. Validating at creation narrows this but cannot close it, because the state can always change before finalization.

Impact

A depositor can escrow proUSD against a tier that is nonexistent, inactive or non-depositable, and can only recover it if the off-chain authority issues a return proof.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/P

roTokenPlusOperations.sol#L92-L106 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L176-L184

Tool Used

Manual Review

Recommendation

Validate the tier before taking custody. Require that it exists, that it is active and that it is depositable in the create path, matching the checks the finalize path already performs, so an obviously unapprovable request cannot be opened in the first place.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/fcdc10b
- Tier is now validated

0xklapouchy

Fix confirmed in fcdc10b.

Issue L-28: The fixed-APR liability is never aggregated on-chain and nothing reserves capital against it [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/62

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`ProTokenPlus` fixes each position's reward at deposit time and stores it on the position, but never sums it. The only aggregate the contract keeps is `totalDepositsBase`, which counts principal alone. `StrategyVault`, which custodies the capital and lends it to the strategist, contains no reference to APR or rewards in its code at all. Every solvency check in the system therefore compares one ledger against another rather than against what is owed.

Vulnerability Detail

`_calculateRewards` derives the reward from the tier APR and duration when a position is created, and it is stored as `lockedRewards`. The figure is immutable from that moment, so the protocol knows its exact obligation per position from day one. That value is read when positions are withdrawn, merged, relocked or split, and accumulated into storage nowhere.

The consequences run through every guard that is supposed to protect capital.

`borrow` is capped by `depositBase`, a principal figure. Nothing is reserved against the reward obligation, so the strategist can draw the entire principal while the protocol owes principal plus rewards. `repay` has no target to satisfy, no maturity and no minimum. Neither function writes a debt record, so the `Borrowed` event is the only trace that capital left, and whether a subsequent repayment covers what was promised is determined entirely off-chain.

Because nothing is tracked, nothing can be reconciled either. `depositBase` in the vault and `totalDepositsBase` in `ProTokenPlus` can drift apart with no on-chain check. A relock of principal the strategist never borrowed illustrates it: `regive` adds the relocked amount to `depositBase` while `ProTokenPlus` retires the original position, so the vault's borrowing ceiling ends up above the recorded obligation. That particular drift is a strategist bookkeeping error he can unwind himself and costs users nothing, which is precisely why it matters here. It is a measurable divergence between two ledgers that no code compares, so it neither reverts nor emits anything.

The missing accumulator is not only bookkeeping. To pay a rate `r` over a term `T` while only a fraction `u` of the capital is actually deployed, the deployed portion must return `r * T / u`, which grows without bound as utilisation falls. Nothing on-chain measures utilisation

or the outstanding liability, so a term that is failing to earn its promised rate produces no signal anywhere until a holder's exit reverts.

Impact

Whether the protocol can meet its fixed-APR promises is not computable on-chain, and no guard prevents the strategist drawing the capital those rewards are owed against.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/state/ProTokenPlusState.sol#L88 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlusOperations.sol#L792-L800 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L311-L312 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L335-L342

Tool Used

Manual Review

Recommendation

Accumulate the liability where it is created. `ProTokenPlus` already computes each reward at deposit, so maintain a running total alongside `totalDepositsBase`, increment it when a position is created or relocked, and decrement it when one is withdrawn or merged away.

Then use it. Bound `borrow` by the capital in excess of that liability rather than by `depositBase` alone, and expose the figure so `claimYield` and any off-chain monitor can compare the reserve against what is actually owed instead of against an earmark that covers only exits already in progress.

Give `borrow` and `repay` a debt record as well, so the amount outstanding, and whether a repayment met the obligation for a given term, are answerable on-chain rather than from the strategist's own accounts.

Discussion

promisque

- Acknowledged

- Liability is tracked off-chain. Capital goes off-chain (MCA → fiat → yield generation)
- Borrow bounded by principal is correct (rewards funded via reserve on repay); no debt record reflects that capital is deployed off-chain where on-chain tracking is impossible
- depositBase and totalDepositsBase are distinct quantities not meant to reconcile
- Solvency of the fixed-APR promise depends on off-chain strategy performance monitored off-chain

0xklapouchy

Acknowledged.

Issue L-29: Moving deposit capital into the withdraw reserve requires a lossy yAsset round trip [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/63

Summary

No function moves value from the deposit pool to the withdraw reserve. `rotate` and `regive` both run the other way. The only route is for the strategist to `borrow` and immediately `repay`, which converts proUSD to yAsset and back through the two `usdCap` clamps and retains less than it started with. That cost lands even when the strategist never intended to deploy the capital and only wants to settle the APR owed on it.

Vulnerability Detail

The deposit pool is a facility the strategist may draw on but is not required to use, while the APR promised to holders accrues on it either way. Settling that obligation means putting value into the withdraw reserve, and the only way in is `repay`.

The two conversions clamp in opposite directions. Redemption divides the USD being converted by `max(price, cap)`, and minting values the returned yAsset at `min(price, cap)`. A round trip therefore retains:

```
min(price, cap) / max(price, cap)
```

With a yAsset trading at 0.97 against a cap of 1.00, drawing 1000 USD of proUSD returns 1000 yAsset units rather than the 1030.9 the live price would give, and repaying those units mints 970 proUSD. The strategist loses 30 to move capital that never left the protocol's control in any economic sense. The loss is zero only while the yAsset sits exactly at its cap, and it compounds if the price moves adversely between the two legs, since each clamp can then bind independently.

Both clamps are individually correct and conservative in the protocol's favour. The defect is that the only path from one internal pool to the other is routed through them, so a purely internal rebalance is charged as though it were an external redemption followed by an unrelated deposit.

Impact

The strategist pays the yAsset's deviation from its cap to settle APR on capital that was never deployed.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L772-L783 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/ProTokenOperations.sol#L798-L809 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/StrategyVault.sol#L405-L408

Tool Used

Manual Review

Recommendation

Add a strategist-only path that moves value from the deposit pool into the withdraw reserve directly, requiring proUSD on top to cover the reward rather than allowing a bare transfer. Moving principal alone would fund rewards out of principal, which is what the existing round trip prevents, so the top-up must be mandatory for the amount moved.

That keeps the property the current design relies on, that the reward has to be supplied from outside, while removing the yAsset conversion from a rebalance that never needed to touch a yAsset.

Discussion

promisque

- Acknowledged
- Both reserves are supposed to be discrete. Rotate's intention was to avoid funds getting stuck in withdraw reserve because of relock
- The finding assumes a forced borrow+repay round trip as the only path to fund the reserve, but the strategist repays an arbitrary amount covering principal and rewards in a single repay call. So there is continuous inbound and outbound movement

0xklapouchy

Acknowledged.

Issue L-30: `FLOOR_TIER_ID` is documented non-depositable but nothing enforces it [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/64

Summary

The constant is declared with the comment "Floor tier ID (non-depositable)", yet its only effect anywhere in the codebase is to exempt tier zero from the rule that a tier must have a non-zero duration. Nothing prevents tier zero from being created as depositable, and no deposit path treats it differently.

Vulnerability Detail

`FLOOR_TIER_ID` appears in exactly three places, all the same expression: `tierId != FLOOR_TIER_ID && duration == 0`, in the initializer, in `addTier` and in `updateTier`. There is no fourth use.

So tier zero can be configured with `duration = 0` and `isDepositable = true`, and the deposit path will accept it, because that path checks `isActive` and `isDepositable` and nothing else about which tier it is. The result is a depositable tier with no lock, and since `_calculateRewards` returns zero whenever the duration is zero, one that pays nothing either. Positions in it still consume `depositCap` headroom and still route capital into the strategist-borrowable pool.

Nothing here is exploitable by a user, and reaching the state needs an admin who configures tier zero contrary to its documented purpose. It is recorded because the constant reads as an enforced restriction and is not one, which is the kind of assumption that survives into an upgrade or a redeployment.

Impact

Tier zero can be made depositable despite being documented as the non-depositable floor tier, producing a zero-duration zero-reward tier that still consumes deposit capacity.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/state/ProTokenPlusState.sol#L14-L15 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/vaults/ProTokenPlus/ProTokenPlus.sol#L285-L300

Tool Used

Manual Review

Recommendation

Enforce the property the comment claims. Reject `isDepositable` on `FLOOR_TIER_ID` in `addTier`, `updateTier` and the initializer, and reject deposits targeting it in the deposit path. If the floor tier is instead meant to be an ordinary tier that merely skips the duration rule, change the comment to say that.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/b5d9e13
- FLOOR_TIER_ID not depositable anymore

0xklapouchy

Fix confirmed in b5d9e13.

Issue L-31: The deposit path has no per-handler failure tolerance, so one unavailable venue blocks the whole allocation [RESOLVED]

Source: https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/issues/65

Summary

The deposit path has no per-handler failure tolerance, so one unavailable venue blocks the whole allocation `_allocateToHandlers` calls `depositYieldAsset` on every configured handler in a single loop with no `try/catch`. If one venue reverts, the entire distribution reverts, including the portions destined for venues that would have accepted them. The withdrawal path already handles this: both `try/catch` blocks in the contract are in `payOut`, and none are on the deposit side.

Vulnerability Detail

`payOut` skips a handler whose `code.length` is zero, wraps `getBalance` in `try/catch`, and wraps `withdrawYieldAsset` in `try/catch` so a crunched venue is passed over. `_allocateToHandlers` has none of that. It approves and calls each handler directly, so any revert propagates.

Aave carries the realistic risk. `ValidationLogic.validateSupply` reverts on `SUPPLY_CAP_EXCEEDED`, `RESERVE_FROZEN`, `RESERVE_PAUSED` and `RESERVE_INACTIVE`, and a stablecoin supply cap filling is a routine condition rather than an exotic one. Morpho Blue is not a comparable risk, being permissionless and uncapped with no pause, so its `supply` fails only on a non-existent market. That would change if the integration moved to MetaMorpho, which does have caps.

Because allocation splits by percentage across all handlers at once, a capped Aave reserve also blocks the share that Morpho would have accepted.

The revert reaches two callers of `_executeMint`:

- `finalizeMintRequest`, so users cannot complete mints for that asset. The yAsset was escrowed at request creation, so the request sits `PENDING` until the condition clears or the authority signs a `PROOF_OF_RETURN`.
- `strategicMint`, which is how `StrategyVault.repay` works. Since `take` draws only on `withdrawProUSD` and `repay` is its only real inflow, a blocked venue can also stall proUSD+ withdrawals unless the strategist repays in a different yAsset.

The transfer to the operations handler happens in the same call immediately before `distributeYAsset`, so a revert unwinds it atomically and no funds are stranded mid-flight.

The condition is recoverable without de-registering anything. `setYProtocolHandlers` validates only that allocations sum to 10000, with no per-entry minimum, so an admin

can pass `[aave, morpho]` with `[0, 10000]`. The `allocationAmount > 0` guard then skips the capped venue while leaving it registered, so its existing balance stays visible to `getYAssetInfo`, `previewPayOut` and `payOut`. This escape requires a second venue to already exist for that asset, since at least one handler must carry the full allocation.

Impact

Minting and strategist repayment for an affected yAsset halt until an admin reroutes the allocation.

Code Snippet

https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/YAssetOperationsHandler.sol#L436-L460 https://github.com/sherlock-audit/2026-07-promis-protocol-july-6th/blob/17ee3b094ca081787d4faf2cd3763d8153bb3bd8/promis-contracts/contracts/core/YAssetOperationsHandler.sol#L244-L267

Tool Used

Manual Review

Recommendation

Give the deposit path the same tolerance the withdrawal path already has. Wrap the per-handler `depositYieldAsset` in `try/catch` and leave any undeployable share as unallocated balance on the operations handler, where the permissionless `distributeUnallocatedYAsset` can place it once the venue reopens. Emit an event on the skipped portion so the condition is visible without waiting for a failed mint to surface it.

Discussion

promisque

- https://github.com/promis-finance/promis-contracts/commit/58a3736
- Per-handler tolerance added

0xklapouchy

Fix confirmed in 58a3736.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.